

# Diseño de Software Orientado a Objetos

---

# Un buen diseño debe considerar...



Independencia de  
componentes



Identificación y  
tratamiento de  
excepciones



Prevención y tolerancia  
de defectos

Cohesión



Acoplamiento

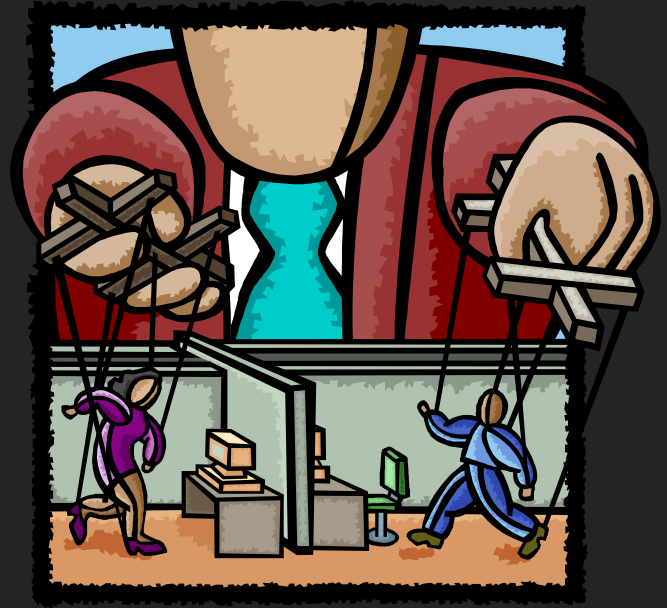


Independencia  
de  
Componentes

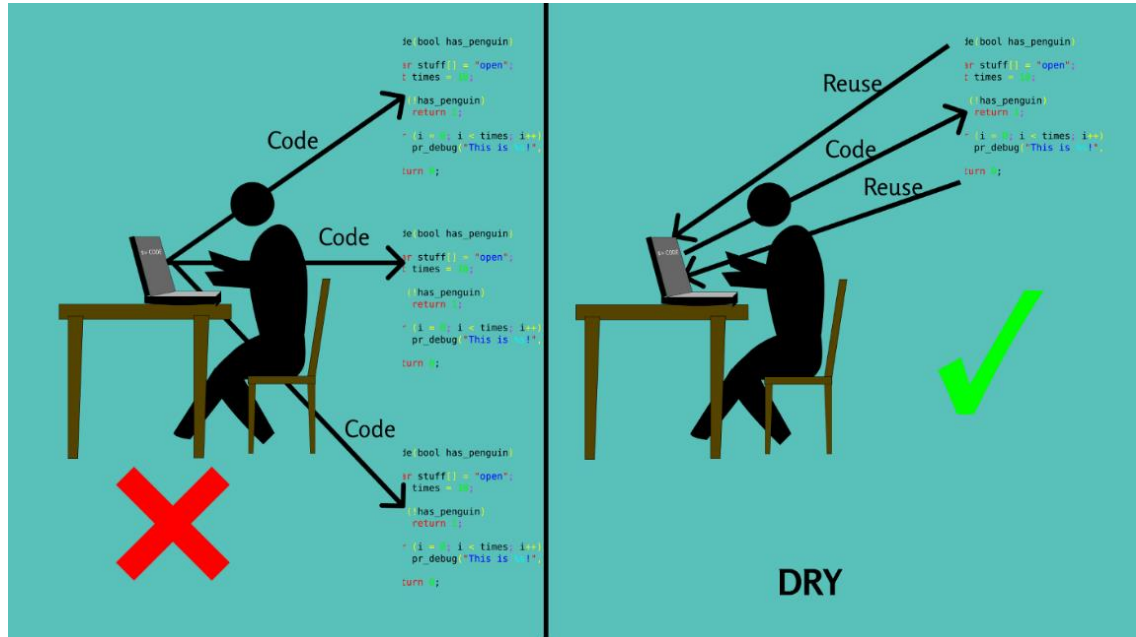


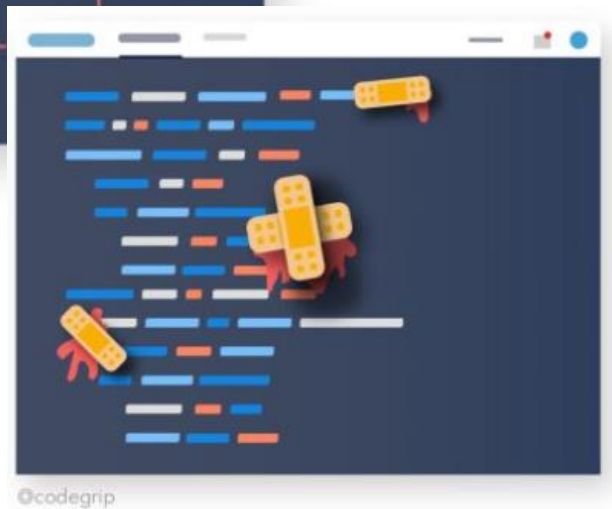
# Principios de Diseño

Para software orientado a objetos



DRY- Don't repeat  
yourself (No te  
repitas)



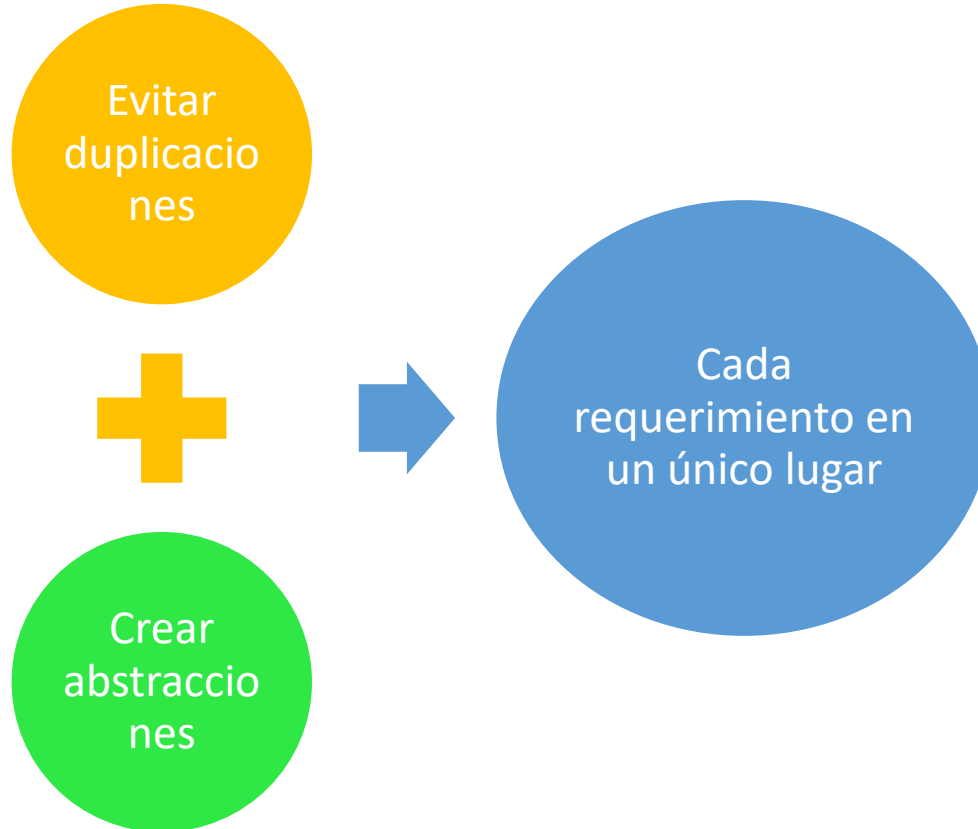


PRINCIPIO DE DISEÑO:  
DRY: Don't repeat yourself !!!!!!!!!!!



@codegrip

# Principio: No te repitas







Principio:  
Separación de  
Intereses  
(Separation  
of Concerns -  
SoC)

Principio de  
Separación  
de Intereses  
(Separation  
of Concerns)

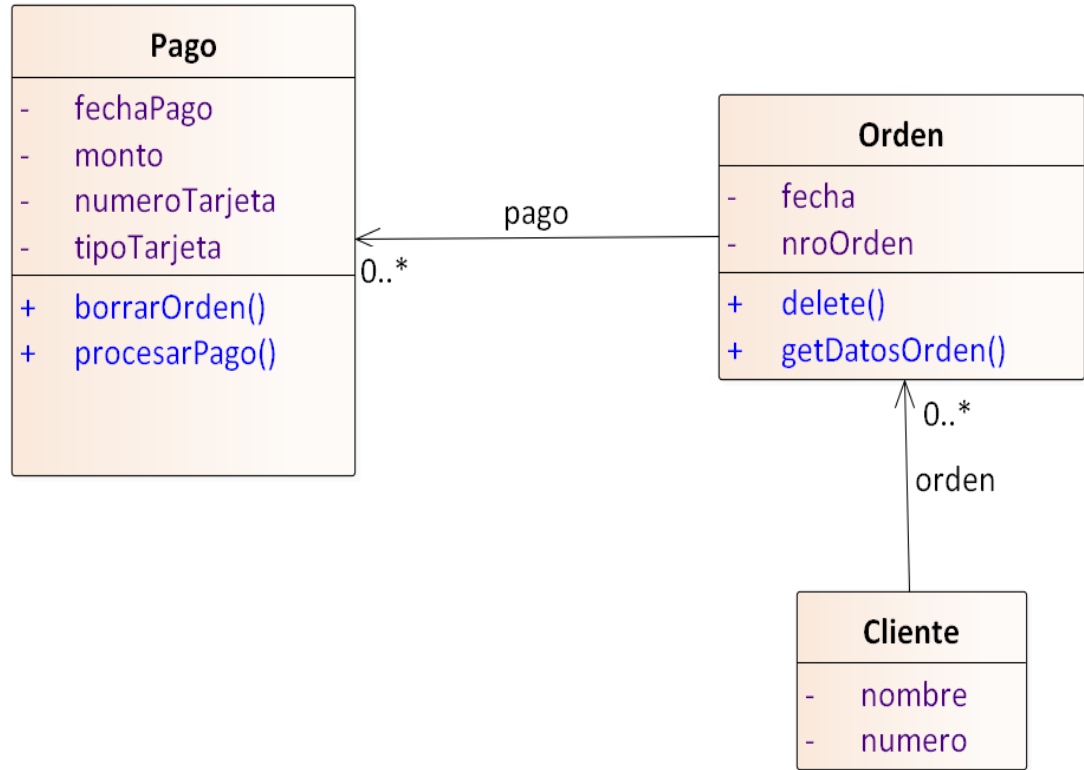
| Pago |                |
|------|----------------|
| -    | cliente        |
| -    | fechaPago      |
| -    | monto          |
| -    | nombre         |
| -    | numeroTarjeta  |
| -    | tipoTarjeta    |
| +    | borrarOrden()  |
| +    | procesarPago() |

## Principio de Separación de Intereses (Separation of Concerns)

- Aislar la aplicación de software en secciones separadas. Cada sección debe abordar una preocupación distinta que tenga poca superposición con otras secciones.
- La separación de intereses se puede aplicar principalmente en 2 niveles:
  - Nivel arquitectónico: separar en varias capas.
  - Nivel de programación: servicios se pueden separar en diferentes niveles.



## Principio de Separación de Intereses (Separation of Concerns)



# Principio de Separación de Intereses (SoC) Beneficios

Gestiona la complejidad y aumenta la mantenibilidad

Mejora la cohesión y reduce el acoplamiento

Facilita la extensión (OCP)

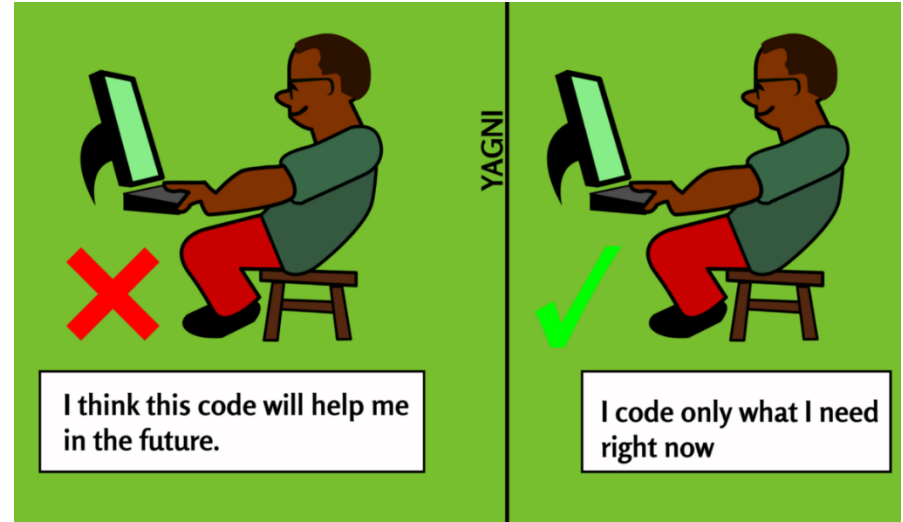
Permite la reutilización

Reduce la fragilidad de los módulos

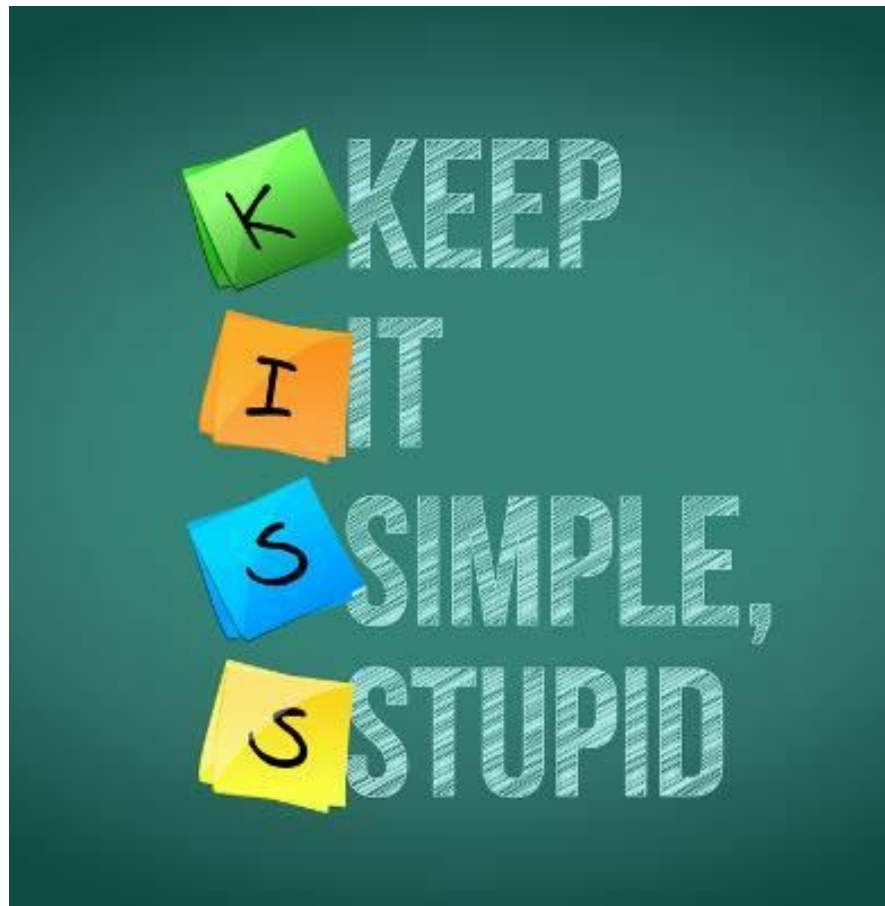
YAGNI – You ain't gonna need it  
(No lo vas a necesitar)

---

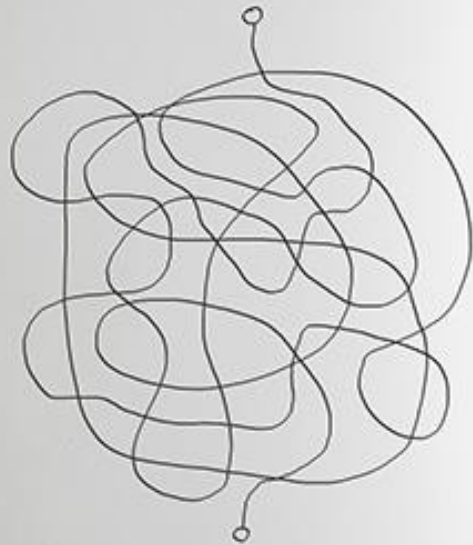
Nunca desarrolle código que crea que será beneficioso en el futuro pero que ahora no tiene valor. El futuro siempre está cambiando. Es una pérdida de tiempo.



## Principio KISS



WRONG STRATEGY



RIGHT STRATEGY





# ¿Qué implica KISS en el software?

- No aumentar infinitamente el nivel de abstracción  $\Rightarrow$  incrementa mucho la complejidad.
- No tiene sentido incluir funciones «en reserva» por si algún día alguien pueda necesitar (aquí el enfoque según el principio YAGNI es más correcto).
- Para qué incluir una librería enorme si solo se necesita un par de funciones de ella.
- La descomposición de algo complejo en componentes simples es un enfoque arquitectónicamente correcto (aquí KISS se hace eco de DRY)
- No siempre se necesita precisión matemática absoluta o detalles extremos: Los datos pueden y deben procesarse con la precisión suficiente para una solución de alta calidad del problema, y los detalles se dan en la cantidad necesaria para el usuario, y no en el volumen máximo posible.

¿Qué implica KISS en el software?

- Mayor calidad del código
- Código más simple de mantener
- Más flexible y fácil de ampliar, modificar, mejorar.

# Principle of least knowledge / Law of Demeter

## Principio de Menor Conocimiento/Ley de Demeter



- Cada unidad debe conocimiento limitado sobre otras unidades; unidades "estrechamente" relacionadas con la unidad actual.
- Cada unidad debe hablar con sus amigos; no hables con extraños. Solo habla con tus amigos inmediatos.

# Principle of least knowledge / Law of Demeter

## Principio de Menor Conocimiento/Ley de Demeter



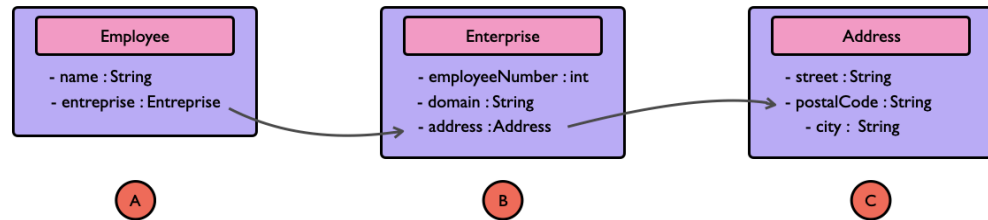
- El método M de un objeto O puede invocar el método de O mismo.
- El método M puede llamar a métodos de cualquier parámetro P.
- El método M puede llamar a objetos creados dentro de M.
- El método M en el objeto O puede invocar métodos de cualquier tipo de objeto que sea un componente directo de O.

DON'T TALK TO STRANGERS

Principle of least  
knowledge / Law  
of Demeter  
Principio de Menor  
Conocimiento/Ley  
de Demeter

The Law of Demeter

**Don't Talk to Strangers**



# Principle of least knowledge / Law of Demeter

## Principio de Menor Conocimiento/Ley de Demeter

- Propósito: producir objetos que estén débilmente acoplados.
- Los objetos fuertemente acoplados requieren una gran cantidad de métodos y requieren una gran cantidad de métodos para funcionar.
- Los objetos deben tener el mínimo de dependencias y saber lo mínimo sobre los otros.
- Ventajas:
  - Reduce la complejidad y aumenta la capacidad de mantenimiento y la adaptabilidad como resultado de reducir el número de métodos que se deben realizar cuando se modifican aspectos del código.
  - Reduce el riesgo de que los métodos no revisados pasen desapercibidos en otra parte del código.
  - Si los objetos están demasiado acoplados y el orden en que los métodos reciben los argumentos se altera, los argumentos deben ser reordenados en cualquier otro lugar donde se llame al método dentro del código.
- Desventajas:
  - Requiere una gran cantidad de métodos de envoltorio para transportar información entre objetos distantes, ya que la información no se puede recuperar de otra manera.

DON'T TALK TO STRANGERS

# Principle of least knowledge / Law of Demeter

## Principio de Menor Conocimiento/Ley de Demeter

- Propósito: producir objetos que estén débilmente acoplados.
- Los objetos fuertemente acoplados dependen en gran medida unos de otros y requieren una gran cantidad de información entre sí para que los métodos funcionen.
- Los objetos débilmente acoplados tienen un número mínimo de dependencias y saben poco sobre los métodos de otras clases.

# Principle of least knowledge / Law of Demeter

## Principio de Menor Conocimiento/Ley de Demeter

- **Ventajas:**

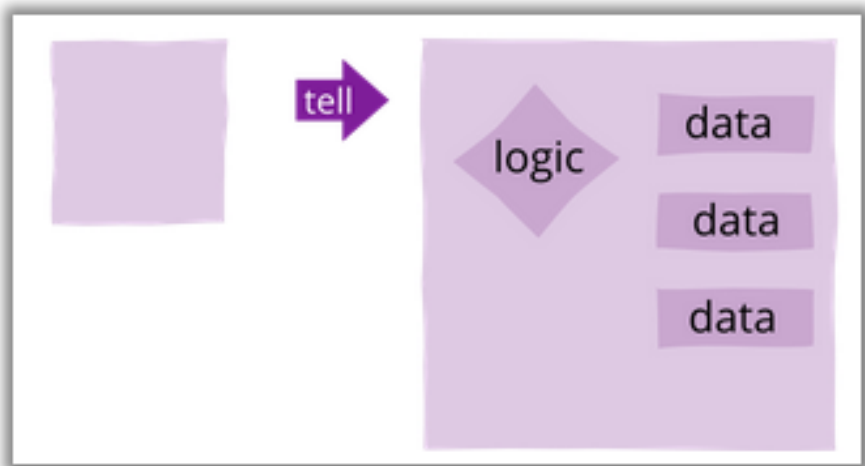
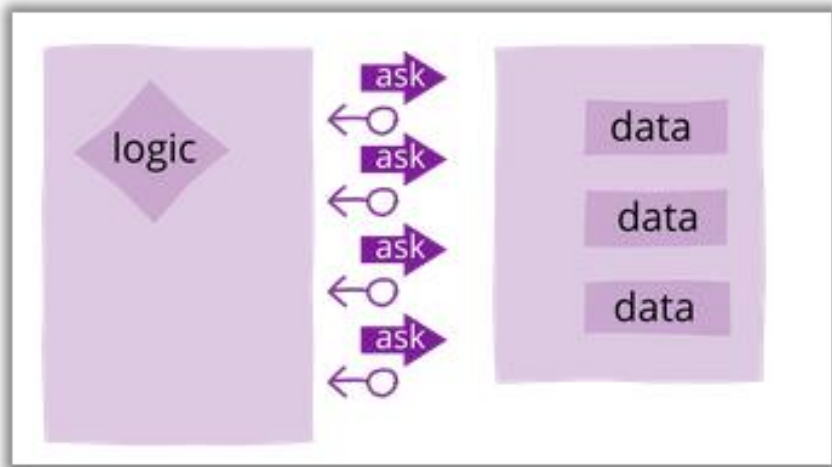
- Mayor legibilidad/cohesión, lo que aumenta la capacidad de mantenimiento y la adaptabilidad.
- Reduce el riesgo de que los métodos no revisados pasen desapercibidos en otra parte del código.
- Si los objetos están demasiado acoplados y el orden en que los métodos reciben los argumentos se altera, los argumentos deben cambiarse en cualquier otro lugar donde se llame al método dentro del código.

- **Desventajas:**

- Requiere una gran cantidad de métodos de envoltorio para transportar información entre objetos distantes, ya que la información no se puede recuperar de otra manera.

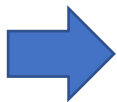


**DON'T**



**DO**

**DON'T**



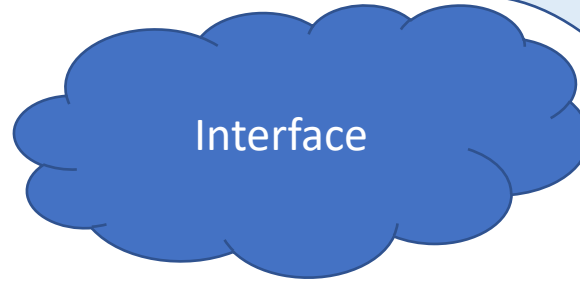
**DO**

PRINCIPIO DE DISEÑO:  
Tell, don't ask !!!!!!!!!

# Tell, don't ask...!!!

(Pedir, no preguntar!!!)



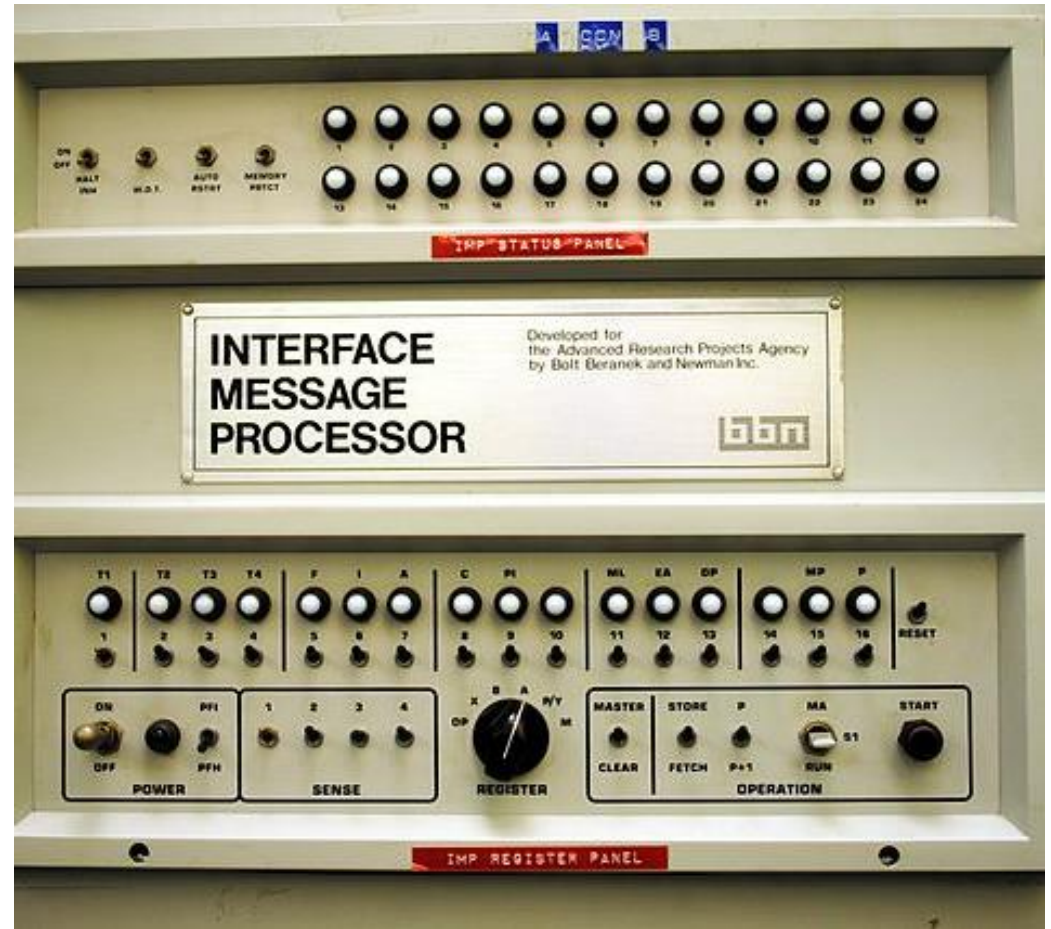


Construcción de código que tiene dos roles:

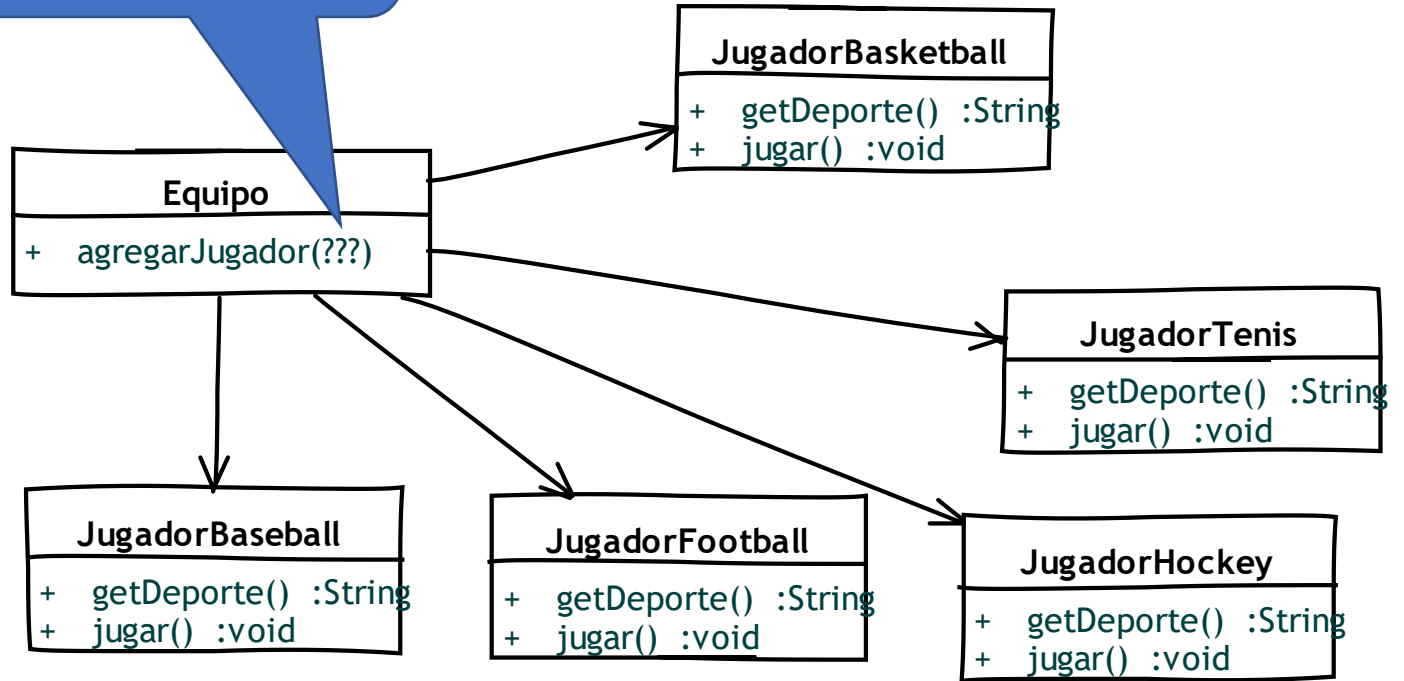
- ➔ Definir comportamiento que aplica a múltiples tipos
- ➔ Hacer foco principal en las clases que usan esos tipos

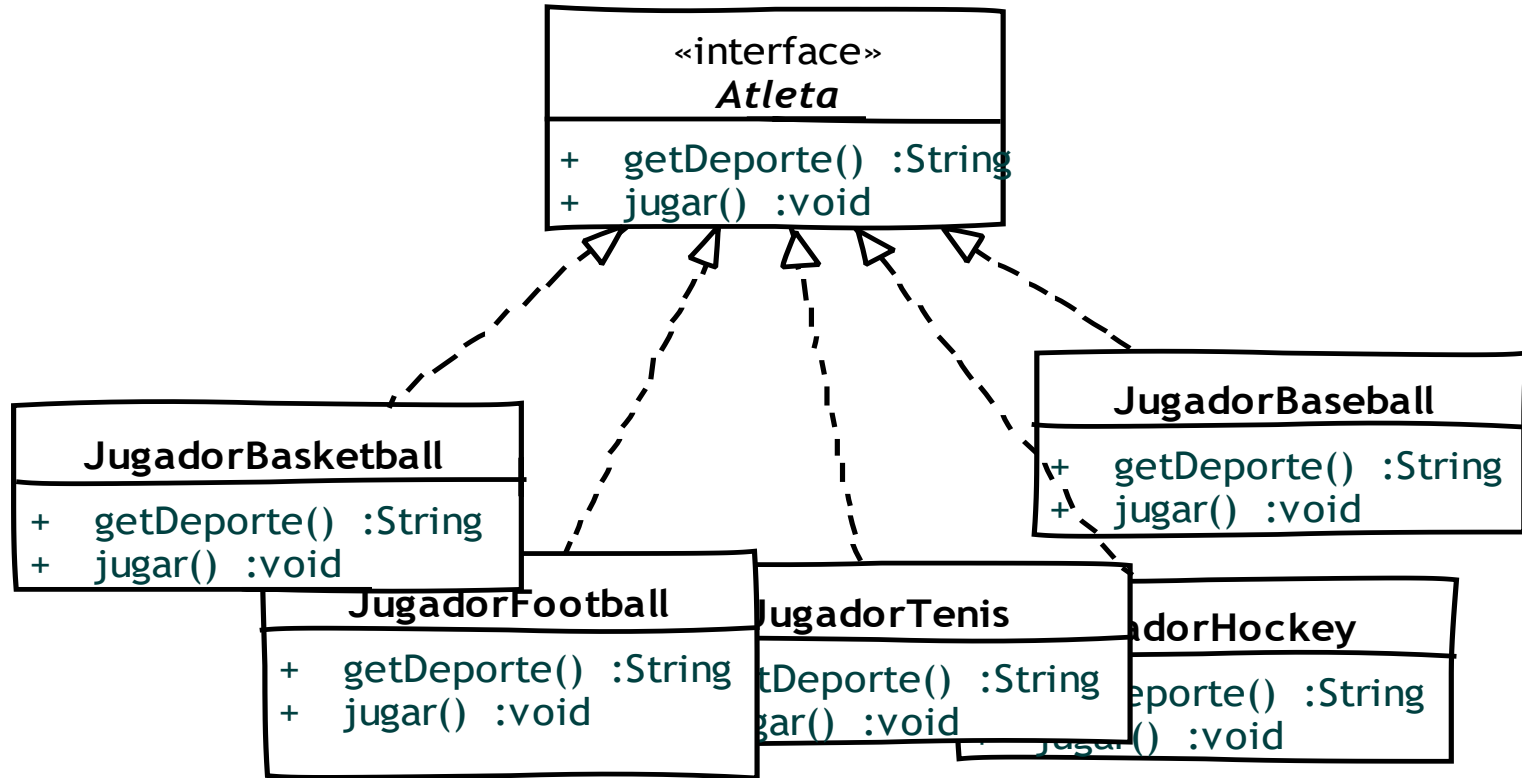
# Program to Interface Principle

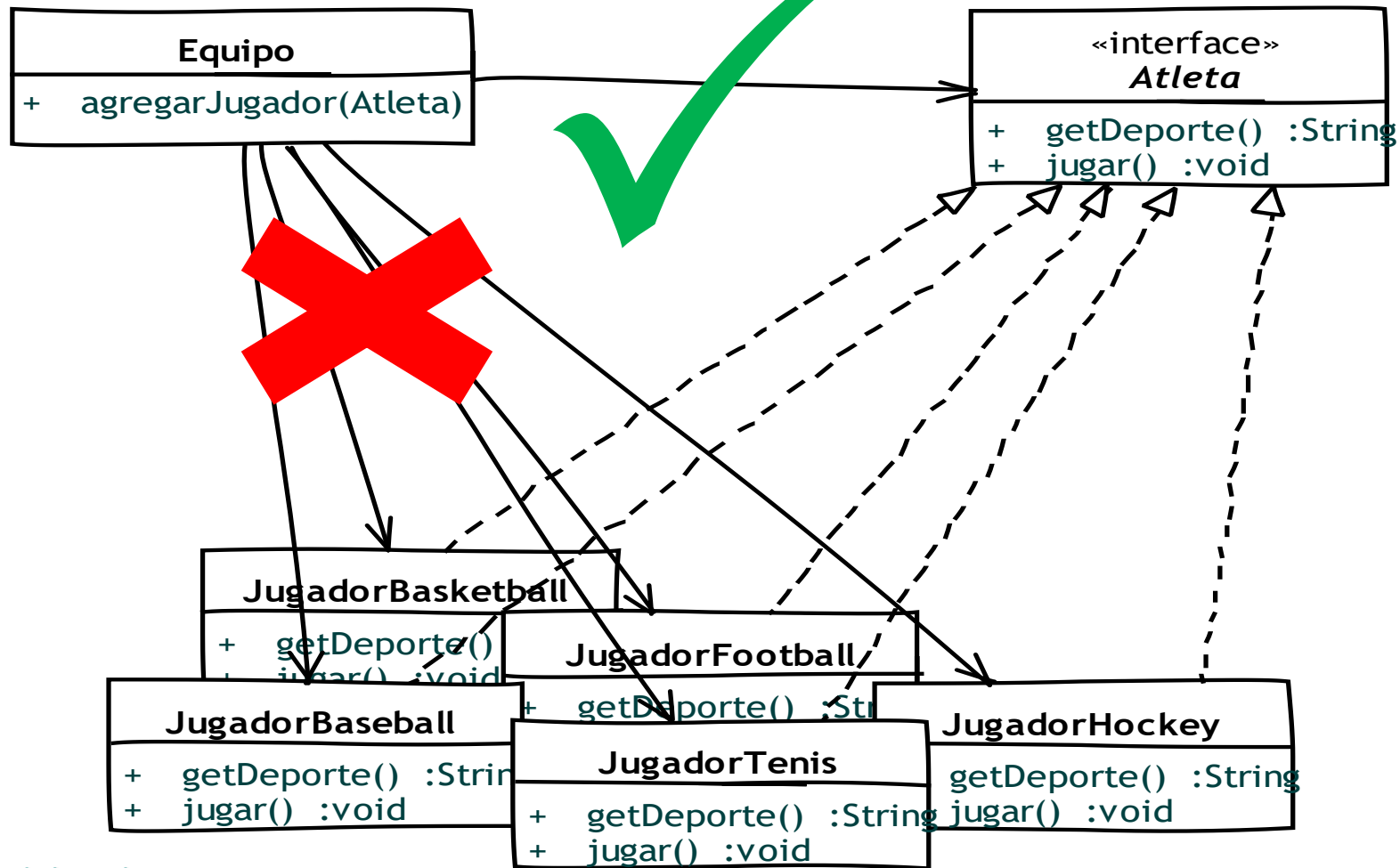
---



¿Qué tipo debería tener este método?









# Trampas con el Principio de Programar hacia la interfaz

#1: Usar una interfaz, pero no programar para una interfaz

#2: Una interfaz para todo

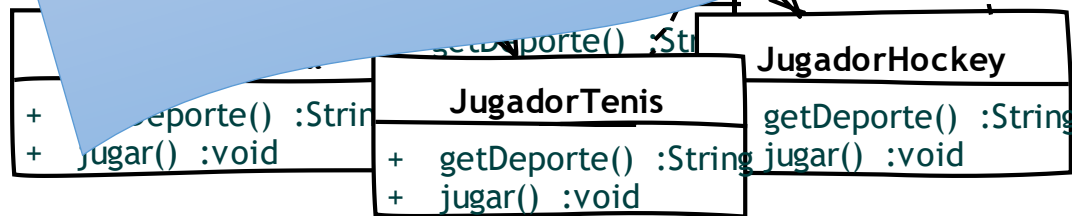
# Dos reglas para pensar

Use interfaces si tiene múltiples implementaciones concretas.

Utilice interfaces (el principio de diseño) para desacoplar su propio sistema de un sistema externo.

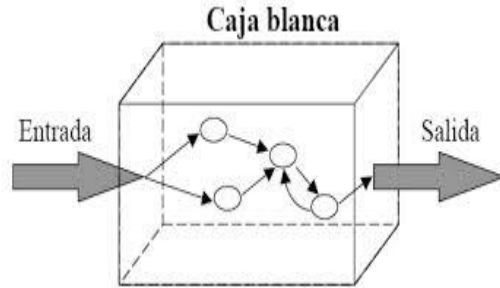


Program to Interface Principle  
Programar hacia la Interfaz y no hacia la implementación!!!



## Herencia

## Composición



Fácil de modificar lo heredado

Rompe el encapsulamiento

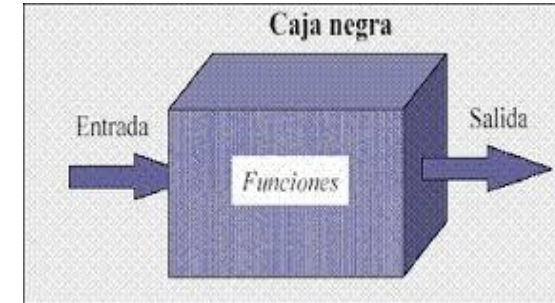
Se define estáticamente

Comportamiento depende de las relaciones

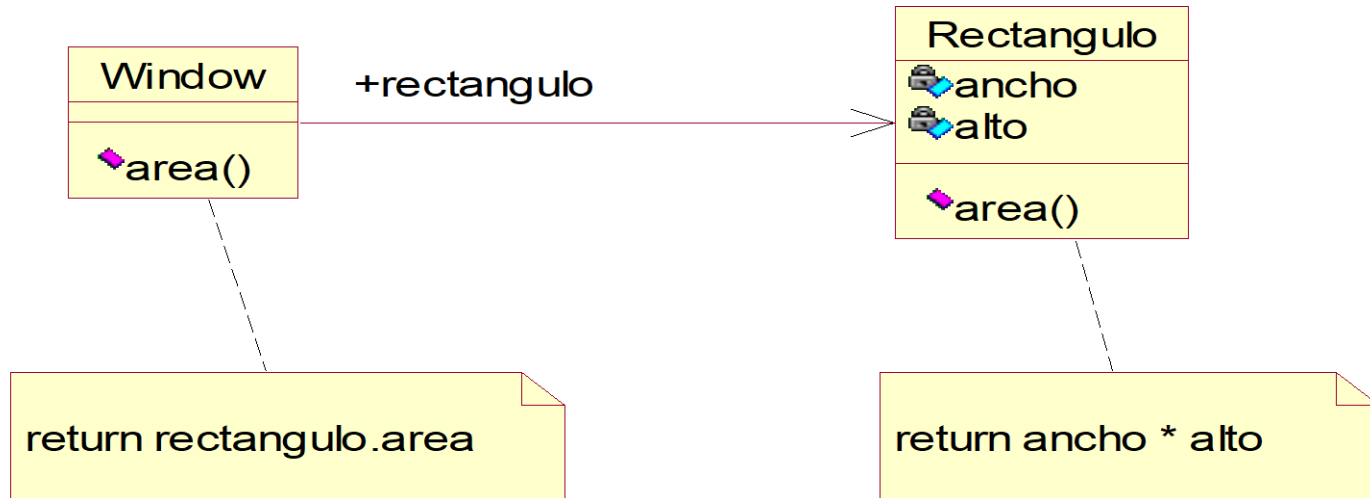
Cambios en tiempo de ejecución

No rompe el encapsulamiento

Se define dinámicamente

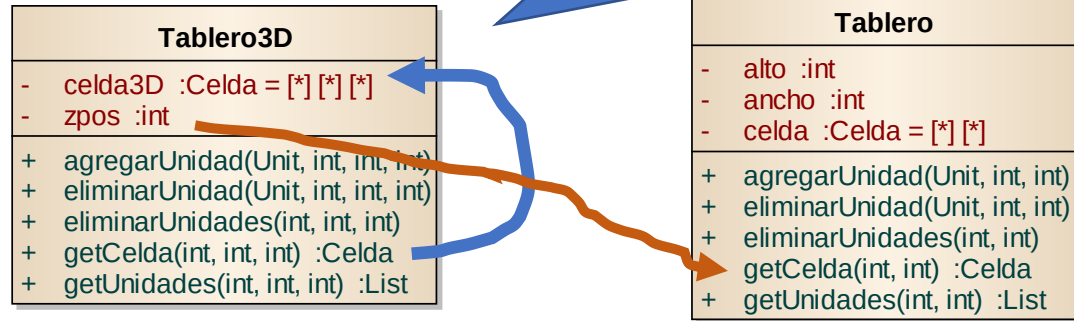
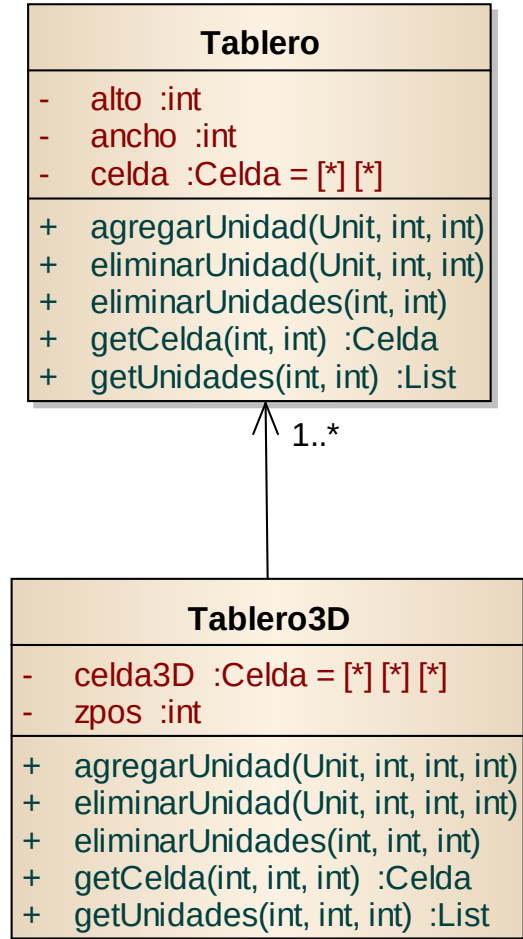


# Delegación



# Composite Reuse Principle

Tablero3D usa la coordenada Z para obtener una instancia de Tablero y luego delega a un método de tablero usando los coordenadas x e y provistas.



Si necesitas usar funcionalidad de otra clase  
sin cambiar esa funcionalidad

## Tablero

- alto :int
- ancho :int
- celda :Celda = [\*] [\*]

- + agregarUnidad(Unit, int, int)
- + eliminarUnidad(Unit, int, int)
- + eliminarUnidades(int, int)
- + getCelda(int, int) :Celda
- + getUnidades(int, int) :List

Tablero3D usa la coordenada Z para obtener una instancia de Tablero y luego delega a un método de tablero usando las coordenadas x e y.

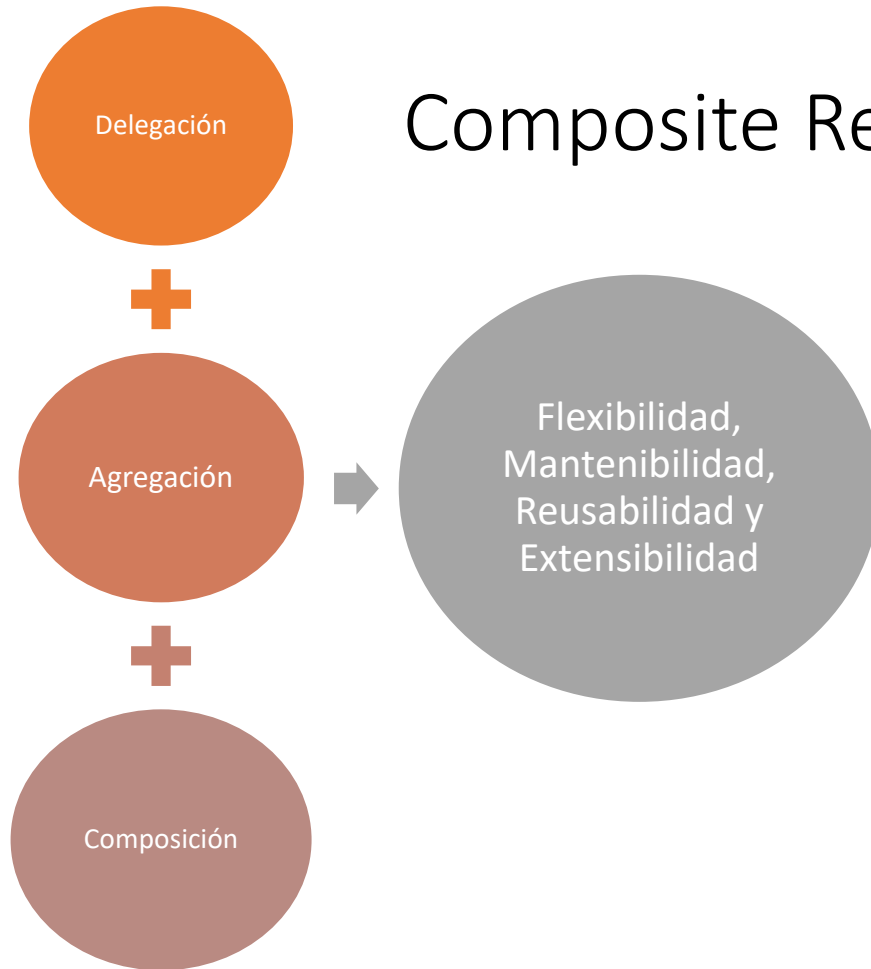


Composite Reuse Principle  
Favorecer la composición sobre la herencia!!!

- + agregarUnidad(Unit, int, int)
- + eliminarUnidad(Unit, int, int)
- + eliminarUnidades(int, int)
- + getCelda(int, int) :Celda
- + getUnidades(int, int) :List

Si necesitas usar funcionalidad de otra clase  
sin cambiar esa funcionalidad

# Composite Reuse Principle



- La delegación es un mecanismo de reuso a nivel de objetos.
- La herencia es un mecanismo de reuso a nivel de clases.



# Principios SOLID

S

Single responsibility principle



O

Open closed principle



L

Liskov substitution principle



I

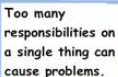
Interface segregation principle



D

Dependency inversion principle

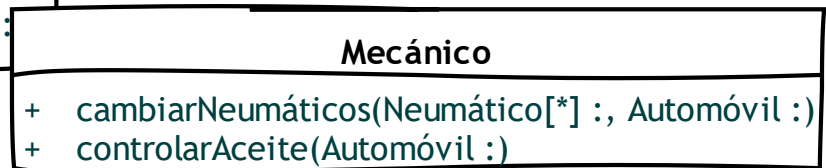
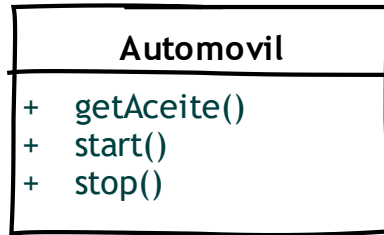




SOLID:  
Single Responsibility  
(Responsabilidad única)

## Automovil

- + cambiarNeumaticos(Tire[\*] :)
- + conducir()
- + controlarAceite()
- + getAceite()
- + lavar()
- + start()
- + stop()



| Automovil                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ <code>getAceite()</code></li> <li>+ <code>start()</code></li> <li>+ <code>stop()</code></li> </ul> |

PRINCIPIO DE DISEÑO:  
Responsabilidad única  
(SRP)

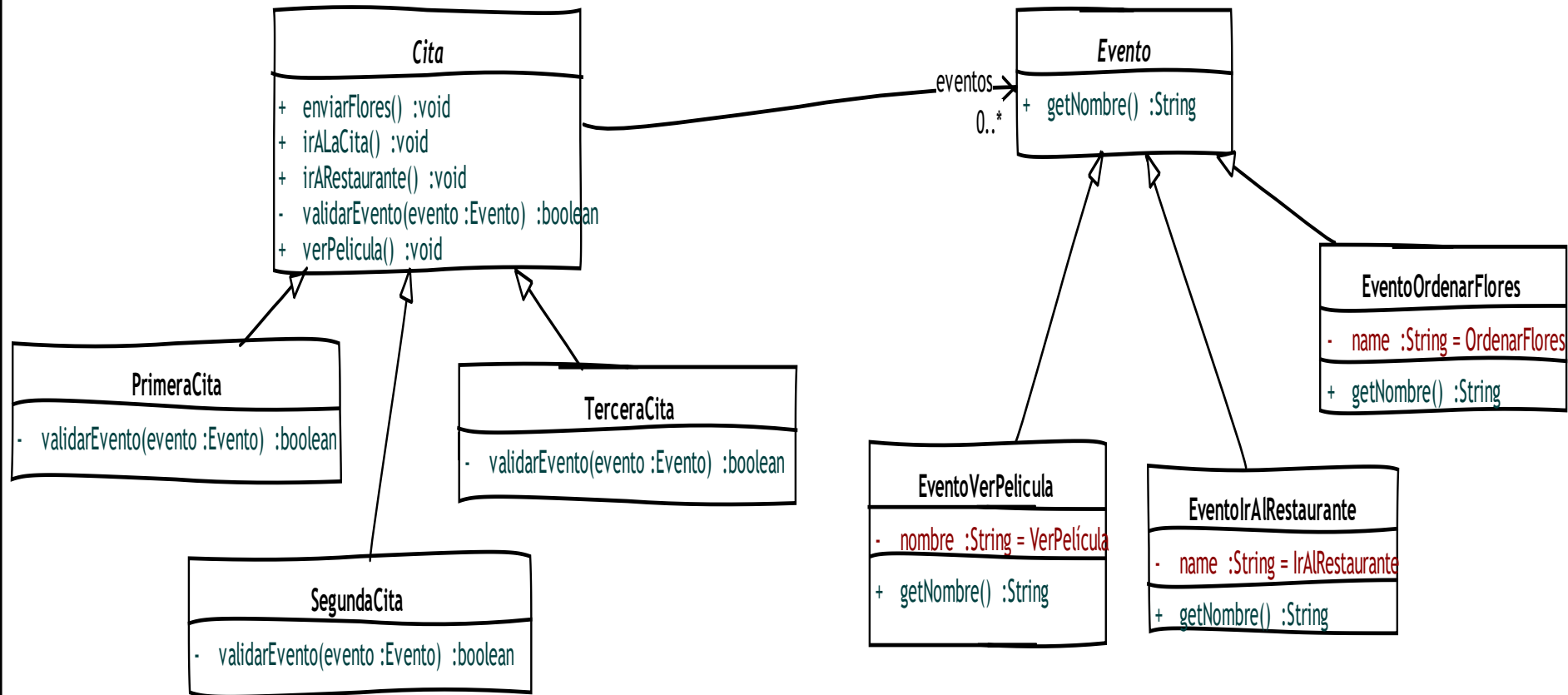


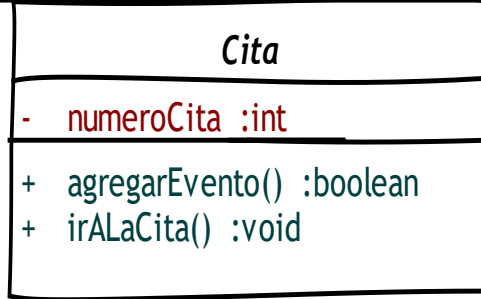
| Mecánico                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ <code>cambiarNeumáticos(Neumático[*] :, Automóvil :)</code></li> <li>+ <code>controlarAceite(Automóvil :)</code></li> </ul> |

# Ejemplo: Clase Automóvil

## ■ Análisis del Principio de responsabilidad única

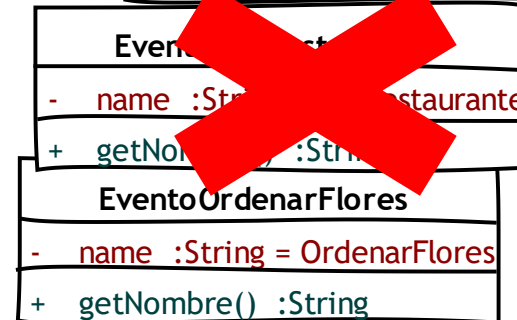
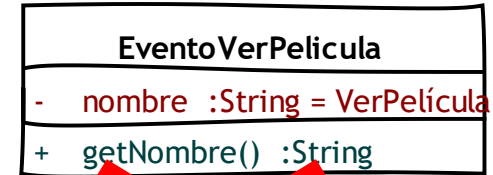
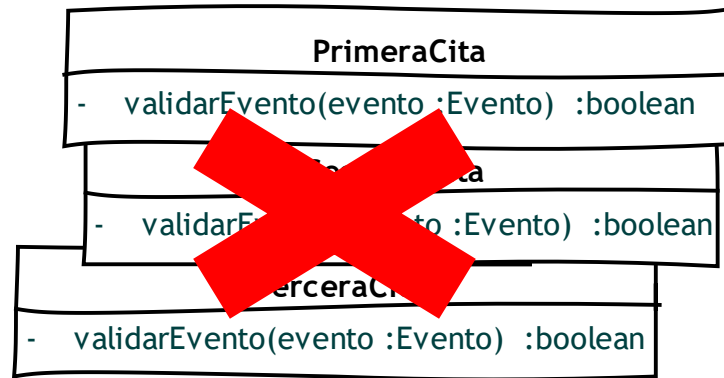
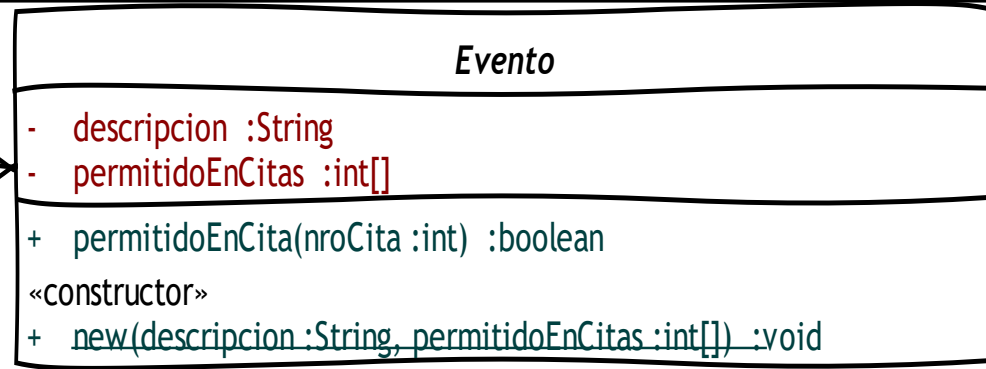
| Responsabilidades                                            | Sigue el Principio                  | Viola el Principio                  |
|--------------------------------------------------------------|-------------------------------------|-------------------------------------|
| El automóvil <i>se inicia</i> a sí mismo                     | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| El automóvil <i>se detiene</i> a sí mismo                    | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| El automóvil <i>se cambia el neumático</i> a sí mismo        | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| El automóvil <i>se conduce</i> a sí mismo                    | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| El automóvil <i>se lava</i> a sí mismo                       | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| El automóvil <i>se obtiene su nivel de aceite</i> a sí mismo | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |





eventos

0..\*





## Cita

- numeroCita :int
- + agregarEvento() :boolean
- + irALaCita() :void

eventos

0..\*

## Evento

- descripcion :String
- permitidoEnCitas :int[]
- + permitidoEnCita(nroCita :int) :boolean
- «constructor»
- + ...

void

SRP + DRY



- nombre :String = VerPelícula
- + getNombre() :String

Evento ...

- name :String = restaurante
- + getNombre() :String

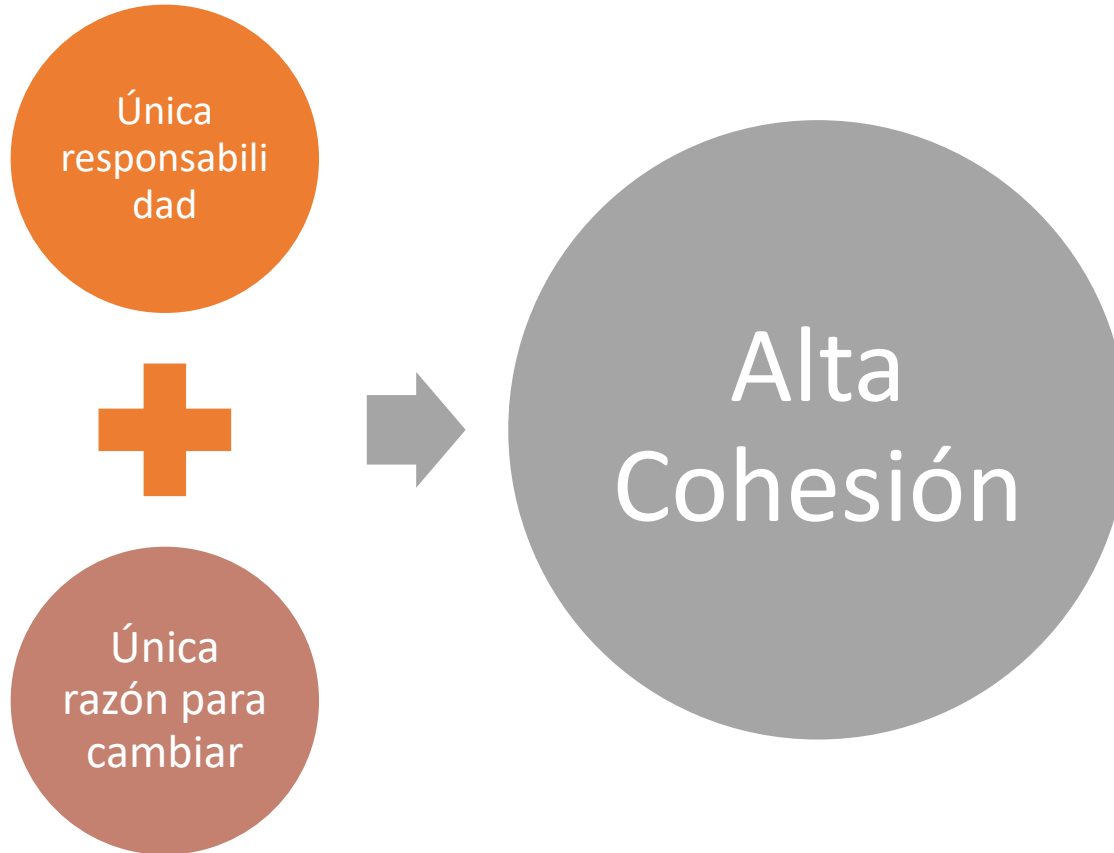
EventoOrdenarFlores

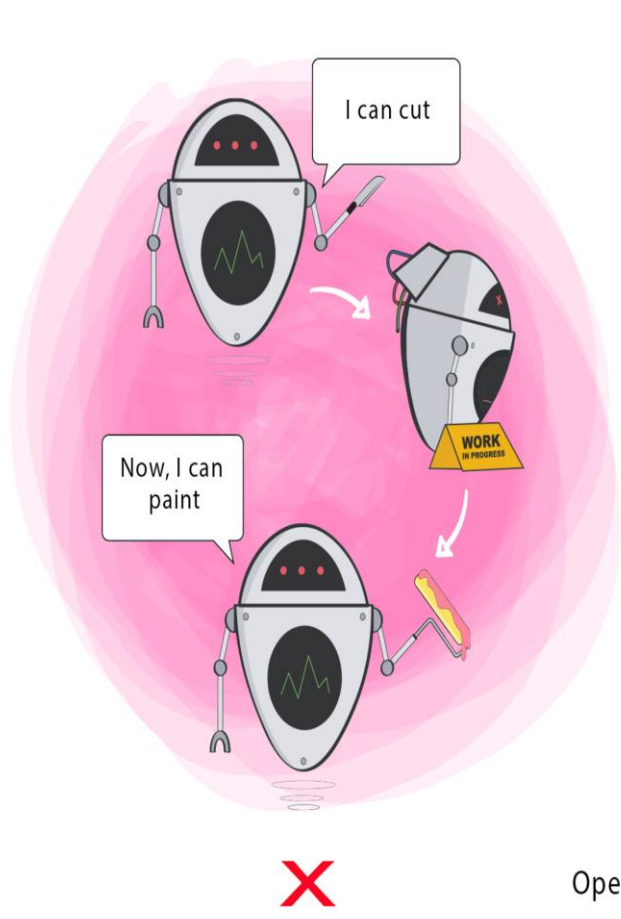
- name :String = OrdenarFlores
- + getNombre() :String



- validarEvento(evento :Evento) :boolean

# SRP: Principio de responsabilidad única



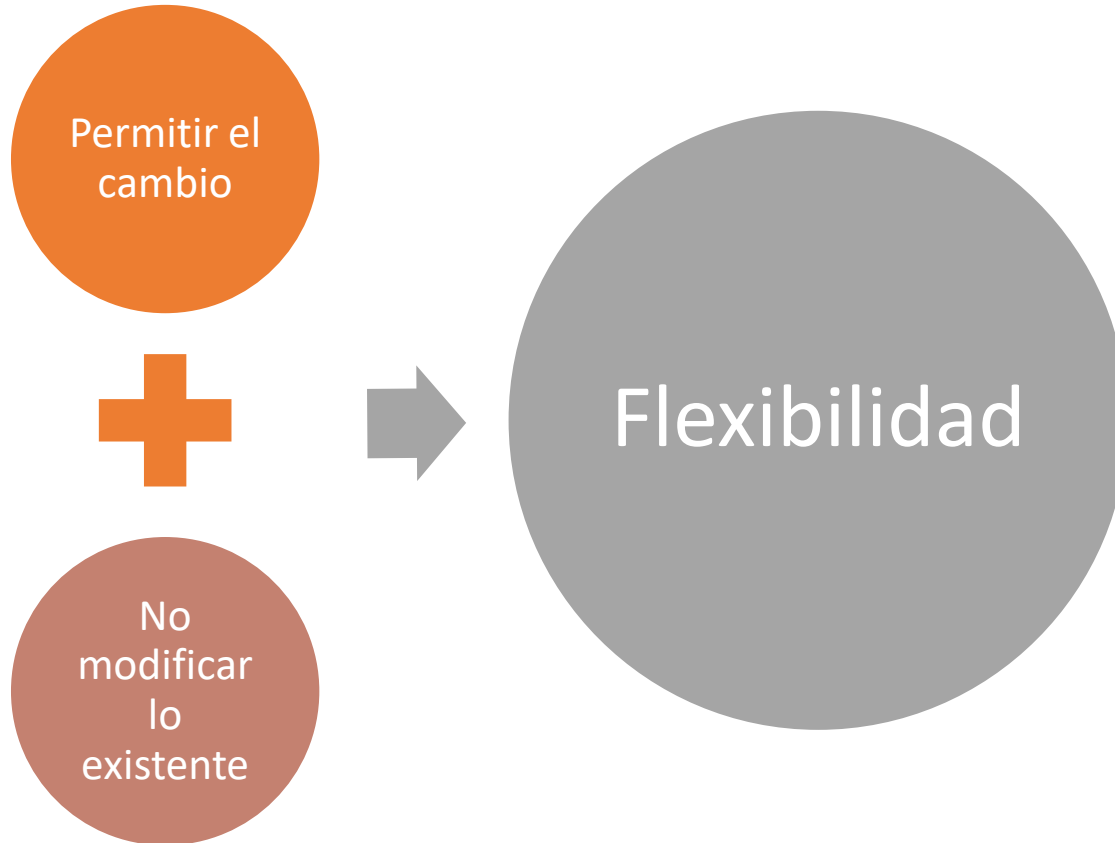


Open-Closed



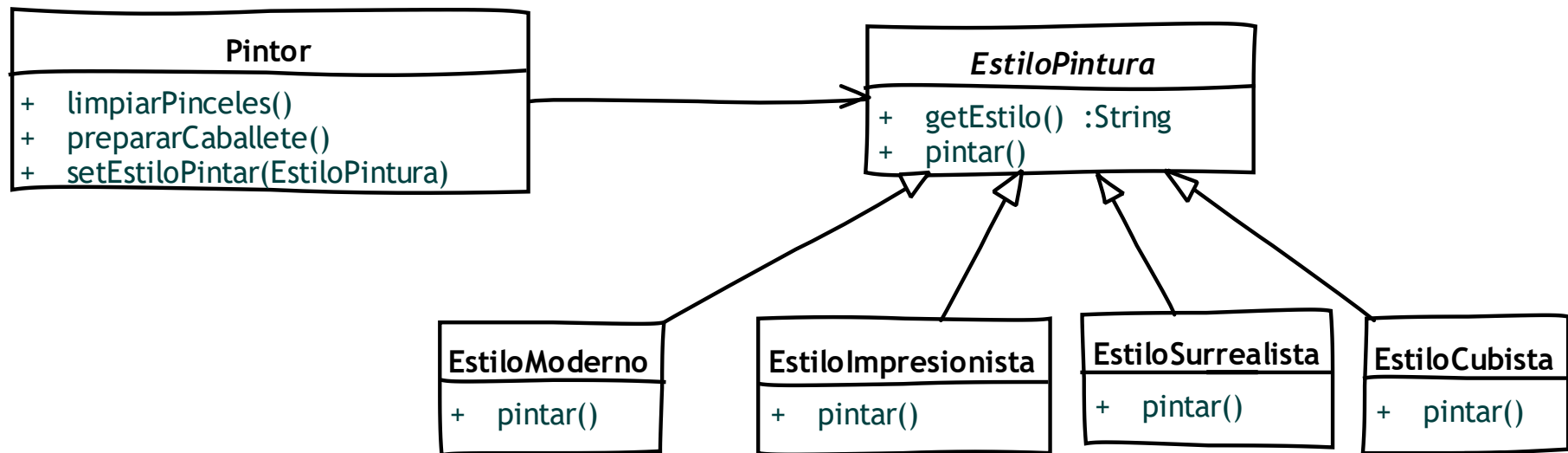
SOLID:  
Open Close  
(Abierto-  
Cerrado)

# OCP- Principio Abierto - Cerrado



## **Pintor**

- + limpiarPinceles() :void
- + pintar() :void
- + prepararCaballete() :void



| Pintor                |
|-----------------------|
| + limpiarPinceles()   |
| + prepararCaballete() |
| + setEstiloPintar()   |

PRINCIPIO DE DISEÑO:  
Abierto – Cerrado /Encapsular lo que varía!!!!!!

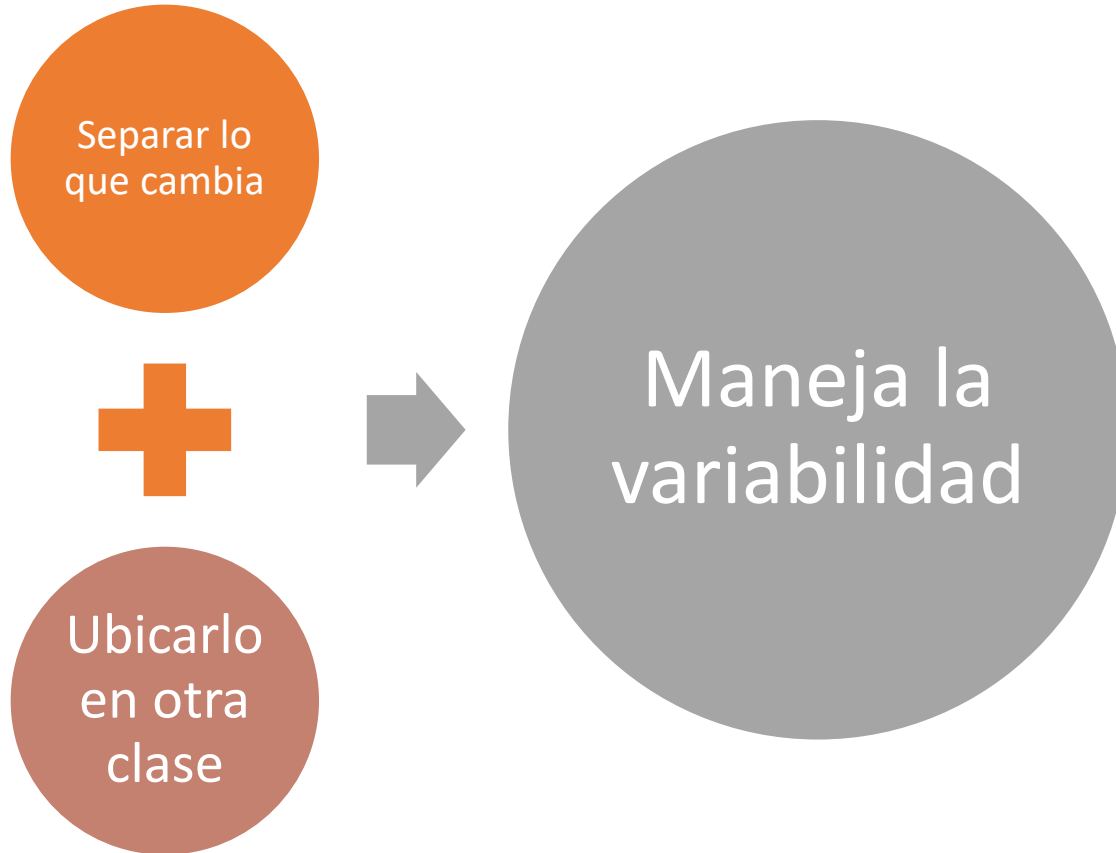


| EstiloImpresionista |
|---------------------|
| + pintar()          |

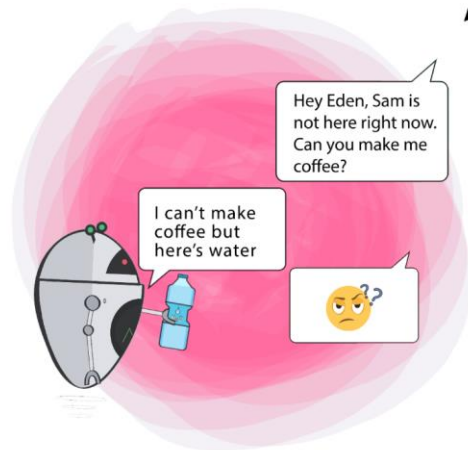
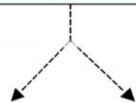
| EstiloSurrealista |
|-------------------|
| + pintar()        |

| EstiloCubista |
|---------------|
| + pintar()    |

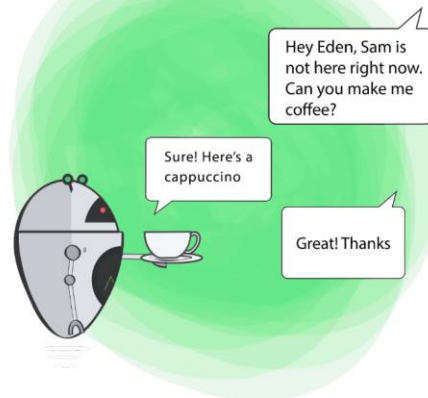
# Principio: Encapsular lo que varía



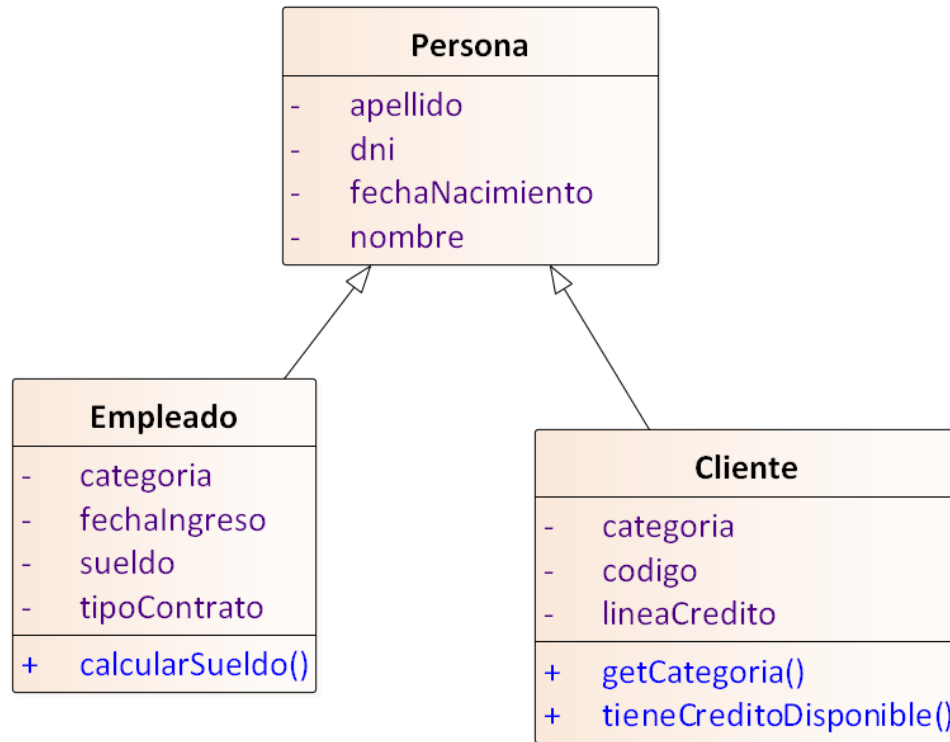




Liskov Substitution

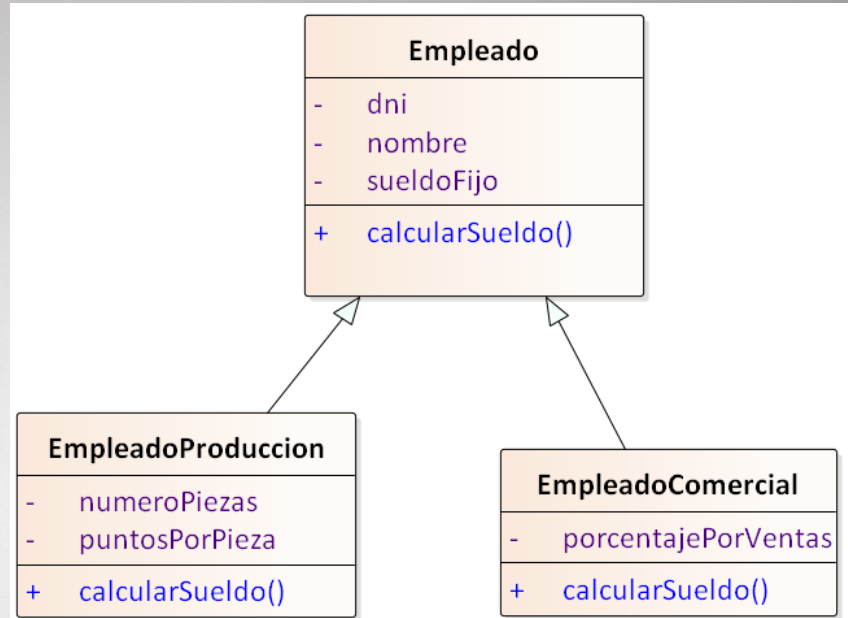


SOLID:  
Liskov  
Substitution  
(Substitution  
de Liskov)



Típico ejemplo inicial de herencia

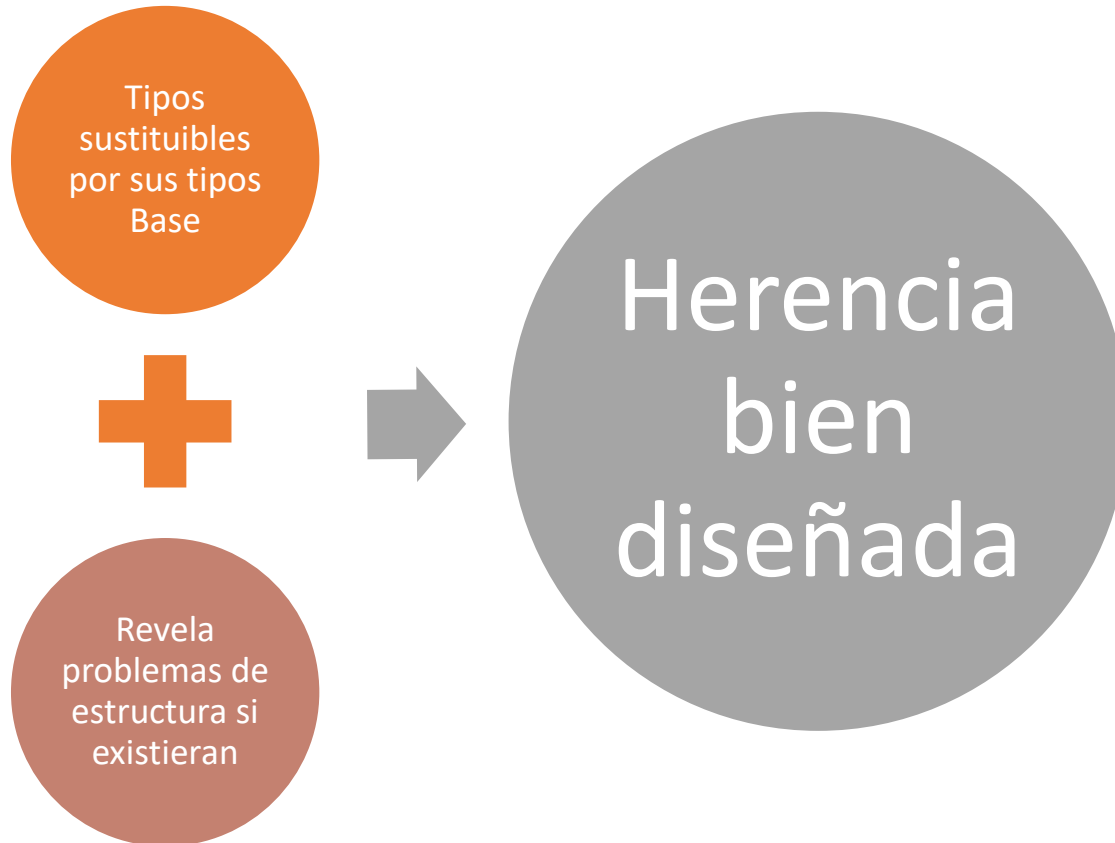




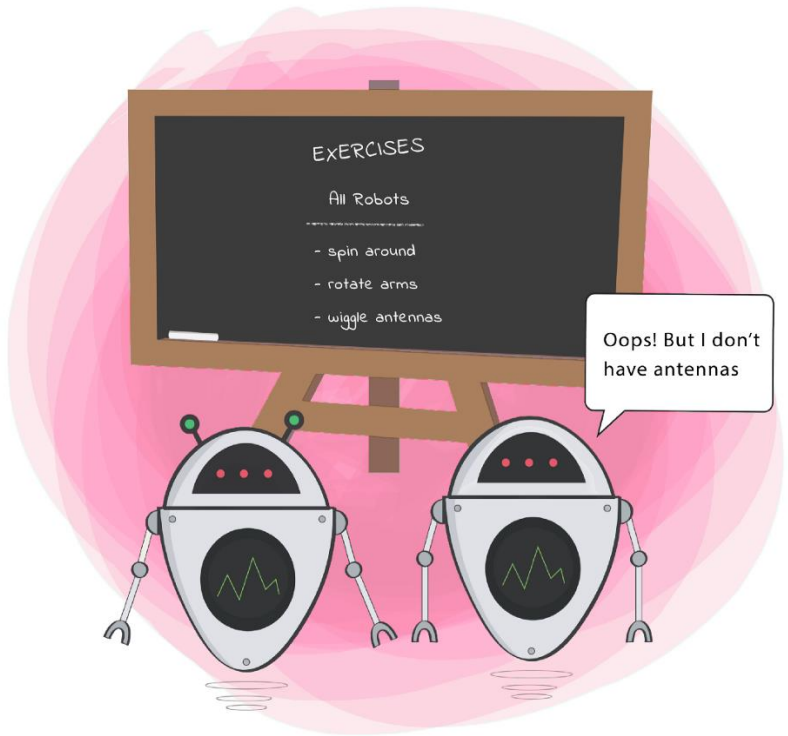
Ejemplo de herencia de comportamiento



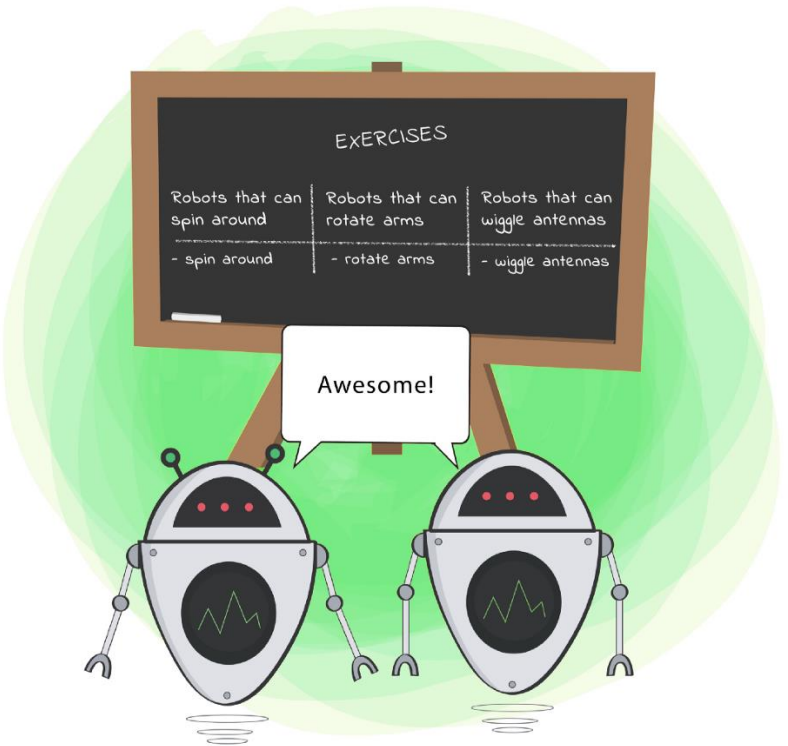
# LSP- Principio Sustitución de Liskov



SOLID:  
Interface Segregation (Segregación de Interfaces)

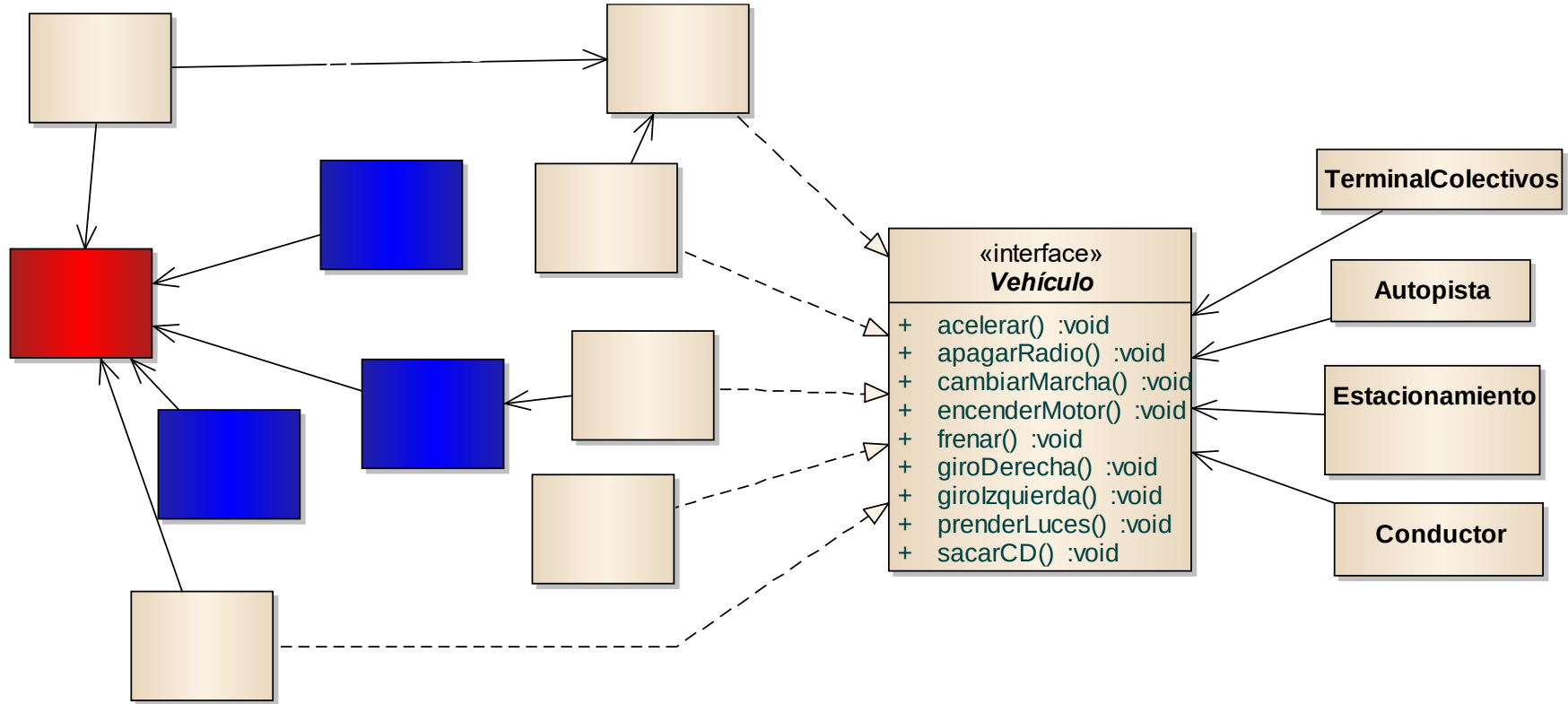


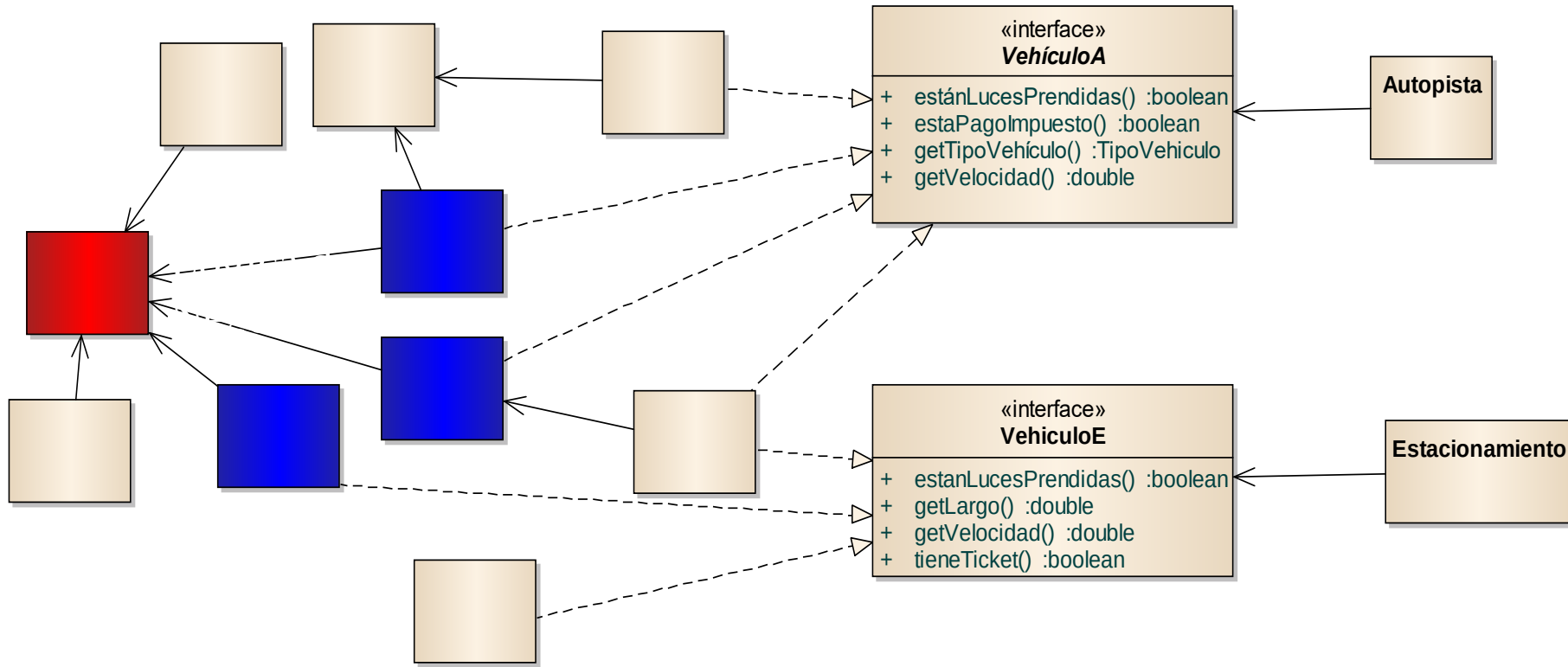
✗



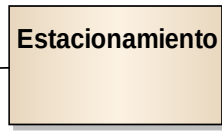
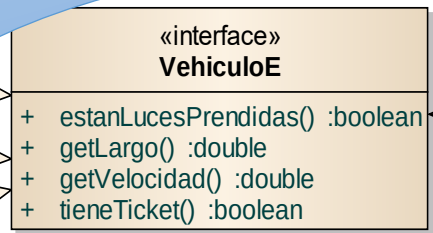
✓

Interface Segregation



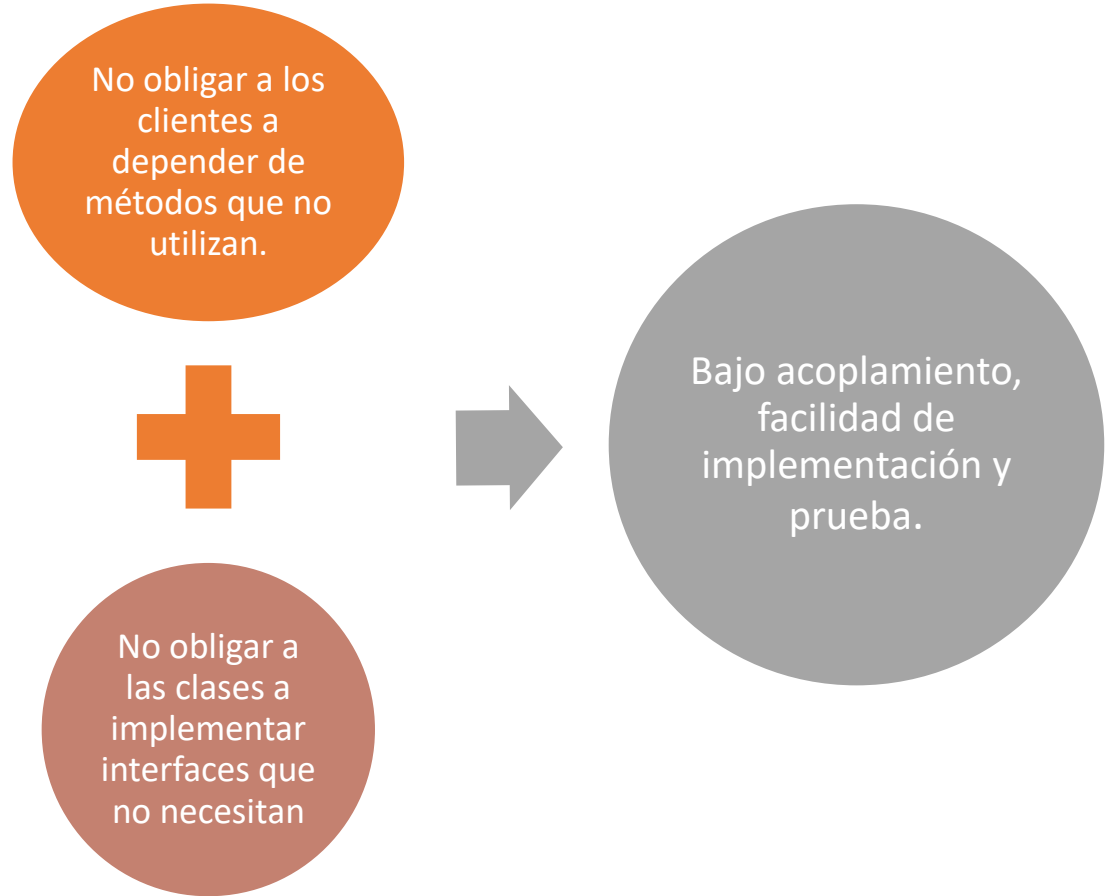


PRINCIPIO DE DISEÑO:  
SEGREGACION DE  
INTERFACES !!!!!!!!



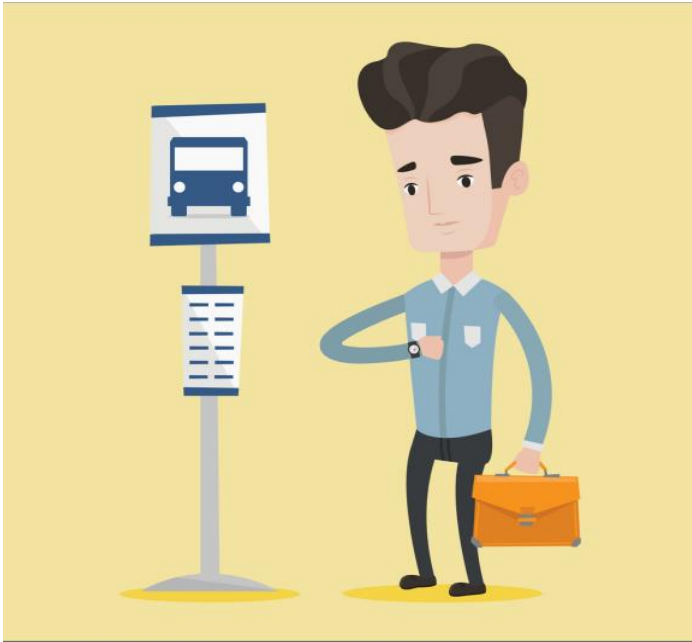


# ISP- Principio Segregación de Interfaces



# SOLID: Dependency Inversion (Inversión de Dependencias)

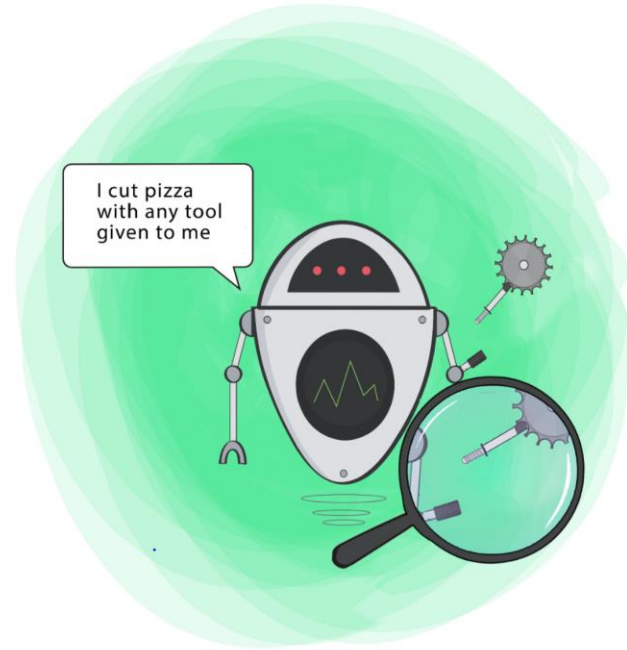
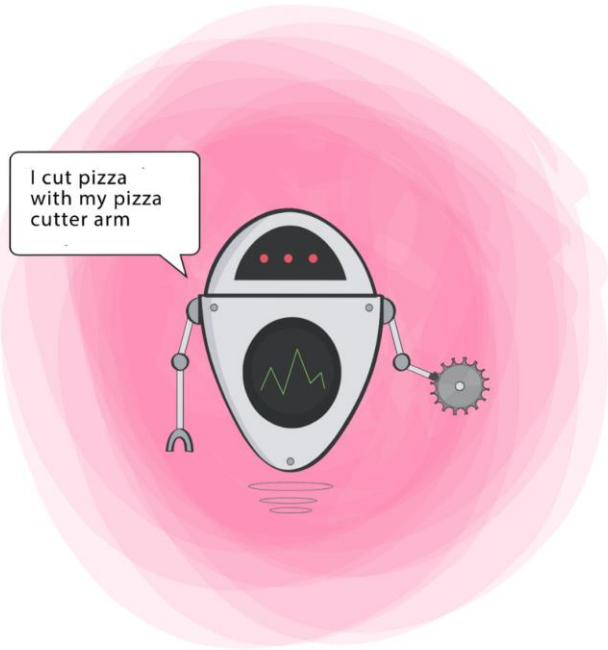
---



# SOLID:

## Dependency Inversion

(Inversión de Dependencias)

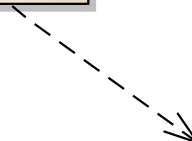


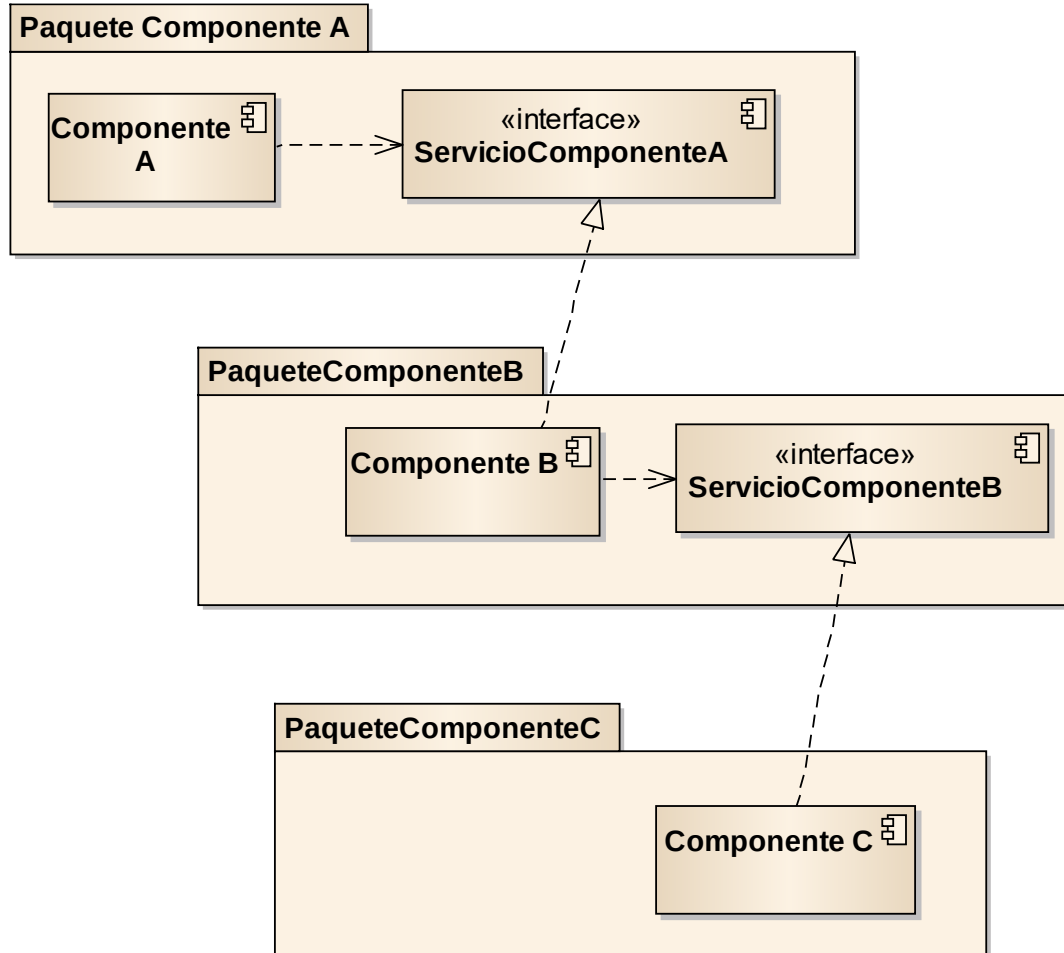
Dependency Inversion

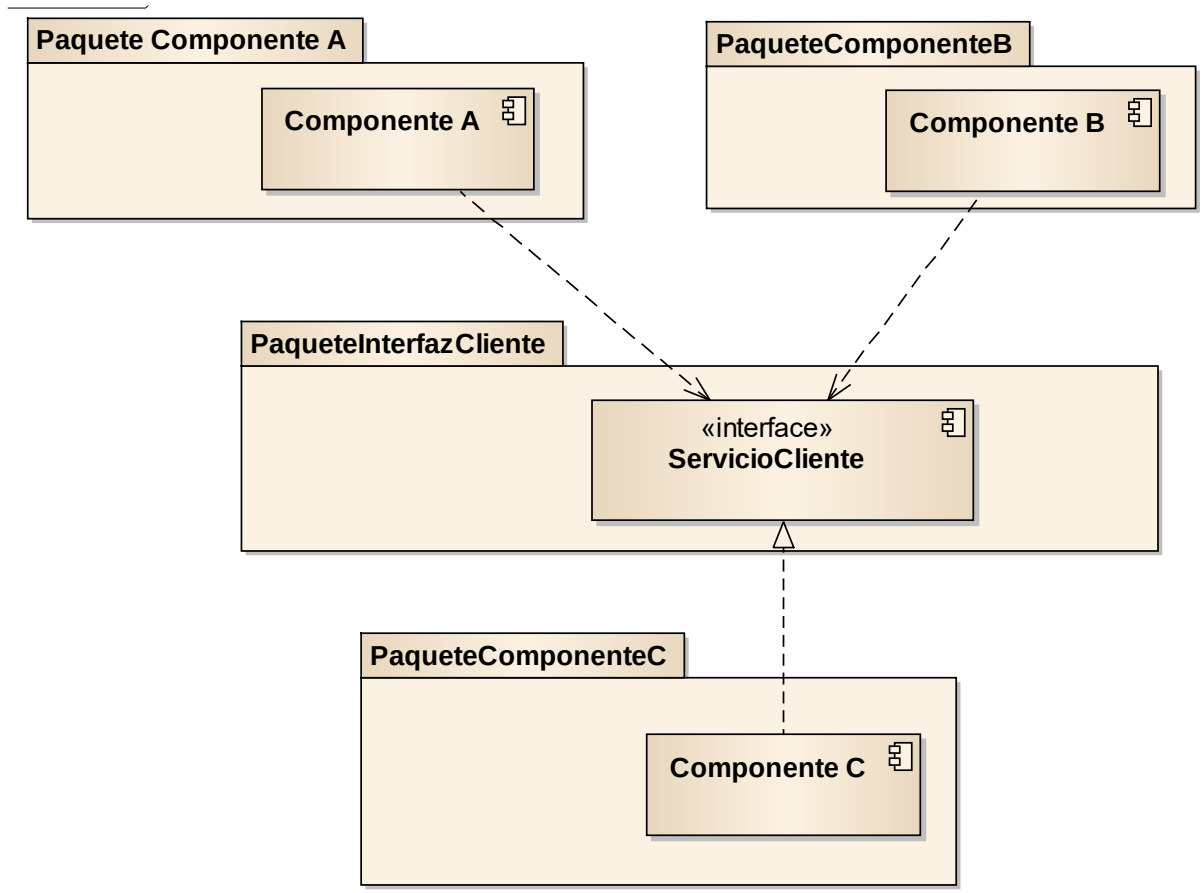
**Componente A** 

**Componente B** 

**Componente C** 









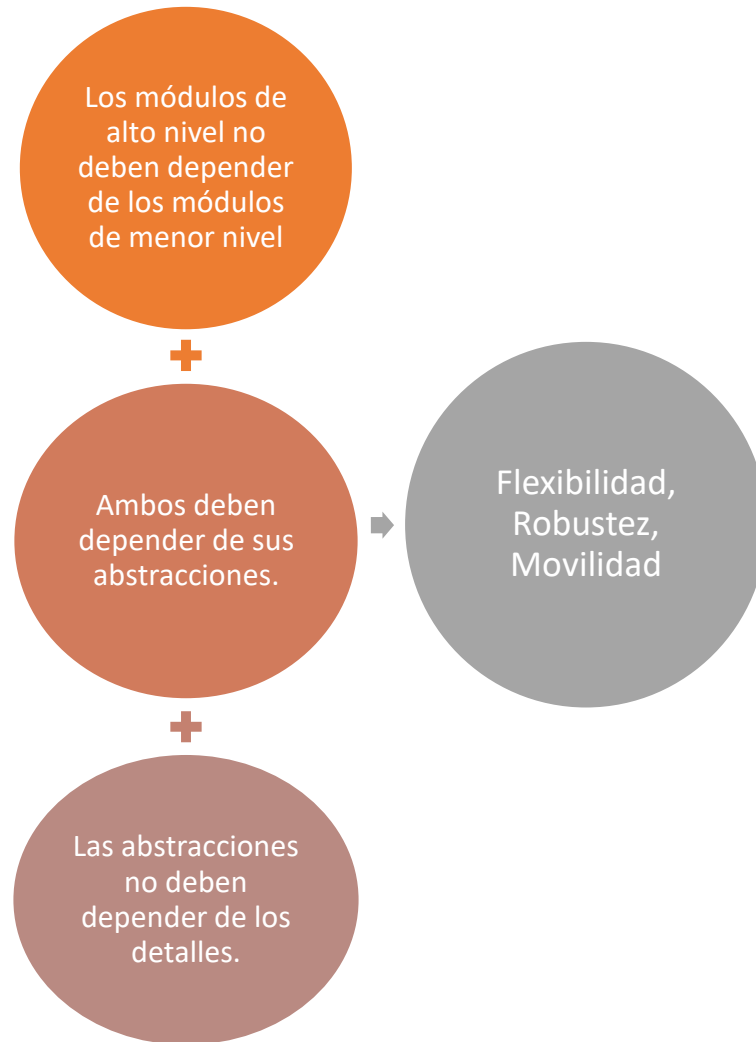
Dependencia ejecución

PRINCIPIO DE DISEÑO:  
Inversión de Dependencias  
(DIP)





# ISP- Principio de Inversión de Dependencias



El desarrollo de Software  
no es un juego de JENGA

**Single Responsibility**

Sólo por que se pueda  
no significa que debas!!!



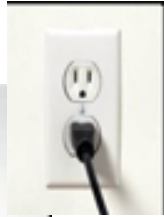
**Open Closed**

La cirugía de cerebro  
no es necesaria para  
usar un sombrero!!!



**Dependency Inversion**

Soldarías una lámpara  
directamente a los cables  
de electricidad en una  
pared??



SOLID



**Interface Segregation**

Quieres que coloque esto?,  
Dónde??



**Liskov Substitution**

Si luce como un pato,  
grazne como un pato,  
pero necesita baterías,  
Tal vez es la abstracción  
equivocada!!!

# Principios de Diseño que vimos...

- Don't Repeat Yourself
- Tell don't ask
- Principle of least knowledge
- Composite Reuse Principle
- Program to Interface Principle
- YAGNI
- KISS

- **SOLID**

- Single Responsibility
- Open Closed (Encapsular lo que varia)
- Liskov Sustitution
- Interface Segregation
- Dependency Inversion

Todos corolarios de  
los principios de  
diseño básicos:



# Principios para Implementación en POO

- Notación CamelCase
- Código Feliz 😊
- Métodos cortos: 7+-2 (siempre y cuando 3 sean comentarios )
- Fácil lectura ➡ Fácil mantenimiento
- Escribir código para quien lo va a mantener.
- Principio del menor asombro: el nombre del método debería corresponder con lo que hace.
- Abrazar el cambio.
- Reusar
- Usar convenciones de código
- En síntesis:



# Bibliografía

- **Design Patterns: Elements of Reusable Object-oriented software.** Gamma, Helm, Johnson, Vlissides
- **Head First Design Patterns.** Freeman & Freeman
- **Head First Object-Oriented Analysis & Design.** McLaughlin, Brett, Pollice Gary & West David.

